



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ
НАУКА У НОВОМ САДУ



Милан Лазаревић

**RepoOptimizer - дистрибуирани
систем за статичку анализу
изворног кода више
програмских језика**

ЗАВРШНИ РАД

Основне академске студије

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Милан Лазаревић	Број индекса:	SV4/2022
Степен и врста студија:	Основне академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Игор Дејановић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

RepoOptimizer - дистрибуирани систем за статичку анализу изворног кода више програмских језика
--

ТЕКСТ ЗАДАТКА:

Пројектовати и имплементирати <i>RepoOptimizer</i>, дистрибуирани систем за аутоматизовану статичку анализу изворног кода више програмских језика (<i>Python, JavaScript, Rust, Java, Go</i>). Архитектуру система засновати на скупу независних микросервиса написаних у програмском језику <i>Rust</i>, међусобно повезаних асинхроном разменом порука путем посредника <i>RabbitMQ</i>, уз примену хибридног модела перзистенције (<i>PostgreSQL, MongoDB и Redis</i>). Омогућити парсирање изворног кода помоћу <i>Tree-sitter</i> библиотеке, детекцију грешака из области безбедности, перформанси, лоших пракси и дуплирања кода уз интеграцију алата <i>Semgrep</i>, као и детерминистичко рангирање озбиљности налаза и шаблонско генерисање конкретних предлога исправки. Приступ систему реализовати кроз веб кориснички интерфејс (<i>React</i>) са визуелним приказом разлика у коду, као и кроз самостални команднолинијски (<i>CLI</i>) алат погодан за интеграцију у <i>CI/CD</i> окружења. Исправност и поузданост решења потврдити јединичним тестовима пословне логике и практичном демонстрацијом рада система над реалним примерима програмског кода. При изради користити препоручену праксу из области софтверског инжењерства. Детаљно документовати решење.
--

Руководилац студијског програма:	Ментор рада:

Примерак за: ☐ - Студента; ☐ - Ментора
--

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	
	КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА	

Редни број, РБР:		
Идентификациони број, ИБР:		
Тип документације, ТД:		Монографска документација
Тип записа, ТЗ:		Текстуални штампани материјал
Врста рада, ВР:		Дипломски - бечелор рад
Аутор, АУ:		Милан Лазаревић
Ментор, МН:		Др Игор Дејановић, редовни професор
Наслов рада, НР:		RepoOptimizer - дистрибуирани систем за статичку анализу изворног кода више програмских језика
Језик публикације, ЈП:		српски/ћирилица
Језик извода, ЈИ:		српски/енглески
Земља публикавања, ЗП:		Република Србија
Уже географско подручје, УГП:		Војводина
Година, ГО:		2026
Издавач, ИЗ:		Ауторски репринт
Место и адреса, МА:		Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)		8/79/10/14/23/0/0
Научна област, НО:		Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:		Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:		статичка анализа кода, Rust, микросервиси, RabbitMQ, AST, Tree-sitter, детекција рањивости, Semgrep
УДК		
Чува се, ЧУ:		У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:		
Извод, ИЗ:		Овај рад представља RepoOptimizer — дистрибуирани систем за статичку анализу кода за шест програмских језика. Реализован кроз Rust микросервиси и RabbitMQ, систем користи Tree-sitter и Semgrep за аутоматску детекцију, рангирање и предлагање исправки за безбедносне пропусте, лоше праксе, дуплирање и перформансне проблеме. Функционалности су доступне преко Web и CLI интерфејса, а рад обухвата опис архитектуре, тестирање и правце даљег развоја.
Датум прихватања теме, ДП:		
Датум одбране, ДО:		10.09.2026
Чланови комисије, КО:	Председник:	Др Гордана Милосављевић, редовни професор
	Члан:	Др Никола Лубурић, ванредни професор
	Члан:	
	Члан, ментор:	Др Игор Дејановић, редовни професор
		Потпис ментора

	UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES 21000 NOVI SAD, Trg Dositeja Obradovića 6	
	KEY WORDS DOCUMENTATION	
Accession number, ANO :		
Identification number, INO :		
Document type, DT :	Monographic publication	
Type of record, TR :	Textual printed material	
Contents code, CC :		
Author, AU :	Milan Lazarević	
Mentor, MN :	Igor Dejanović, Phd., full professor	
Title, TI :	RepoOptimizer - A distributed system for static analysis of source code in multiple programming languages	
Language of text, LT :	Serbian	
Language of abstract, LA :	Serbian/English	
Country of publication, CP :	Republic of Serbia	
Locality of publication, LP :	Vojvodina	
Publication year, PY :	2026	
Publisher, PB :	Author's reprint	
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6	
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	8/79/10/14/23/0/0	
Scientific field, SF :	Electrical and Computer Engineering	
Scientific discipline, SD :	Applied computer science and informatics	
Subject/Key words, S/KW :	static code analysis, Rust, microservices, RabbitMQ, AST, Tree-sitter, vulnerability detection, Semgrep	
UC		
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad	
Note, N :		
Abstract, AB :	This paper presents RepoOptimizer, a distributed static code analysis system for six programming languages. Implemented via Rust microservices and RabbitMQ, it leverages Tree-sitter and Semgrep to automatically detect, rank, and suggest fixes for security vulnerabilities, code smells, duplication, and performance issues. Accessible through Web and CLI interfaces, the paper outlines the system's architecture, testing approach, and future development directions.	
Accepted by the Scientific Board on, ASB :		
Defended on, DE :	10.09.2026	
Defended Board, DB :	President: Gordana Milosavljević, Phd., full professor Member: Nikola Luburić, Phd., assoc. professor Member: Member, Mentor: Igor Dejanović, Phd., full professor	Mentor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
1.1	Циљ рада	1
1.2	Пристап и обим рада	1
1.3	Структура рада	2
2	Преглед постојећих решења	3
2.1	SonarQube	3
2.2	Semgrep	3
2.3	CodeQL	4
2.4	Упоредни преглед и позиционирање решења	4
3	Коришћене технологије	7
3.1	Rust	7
3.2	Backend технологије	7
3.3	Обрада и размена података	8
3.4	Frontend технологије	9
3.5	DevOps и квалитет кода	9
4	Архитектура система	13
4.1	Преглед архитектуре	13
4.2	Микросервиси	14
4.3	Analysis сервис — категорије правила	15
4.4	Конкурентна обрада фајлова	16
4.5	Ток података кроз систем	16
4.6	Праћење статуса послова и историја анализа	17
4.7	Безбедносни аспекти архитектуре	17
5	Имплементација	19
5.1	Auth Service	19
5.2	Parser Service	23
5.3	Analysis Service	27
5.4	ML Ranker Service	32
5.5	Suggestion Generator Service	37
5.6	API Gateway	41
5.7	Веб кориснички интерфејс	46
5.8	CLI алат	48
6	Тестирање	51
6.1	Пристап тестирању	51
6.2	Шаблон тестних података (test fixtures)	52

6.3	Гранични (boundary) тестови	52
6.4	Регресиони тестови	53
6.5	Тестирање модела рангирања	54
6.6	Ограничења приступа тестирању	55
7	Демонстрација	57
7.1	Припрема окружења	57
7.2	Пријава	57
7.3	Покретање анализе	58
7.4	Праћење напретка	59
7.5	Преглед резултата и предлог исправке	59
7.6	Историја анализа	60
7.7	Анализа путем CLI алата	61
8	Закључак	63
8.1	Преглед урађеног	63
8.2	Предности решења	63
8.3	Ограничења решења	64
8.4	Правци даљег развоја	65
	Списак слика	67
	Списак листинга	69
	Списак табела	71
	Списак коришћених скраћеница	73
	Списак коришћених појмова	75
	Биографија	77
	Литература	79

Глава 1

Увод

Квалитет изворног кода директно утиче на одрживост, безбедност и поузданост софтверских система. Ручна ревизија кода (code review) је и даље један од најпоузданијих начина да се проблеми у коду открију пре него што доспеју у продукцију, али је истовремено временски захтевна и зависи од искуства и пажње особе која ревизију спроводи. Са растом обима пројеката и броја програмских језика које један тим користи, потреба за аутоматизованим алатима који унапред и доследно указују на проблематичне делове кода постаје све израженија.

Постојећи алати за статичку анализу кода (поглавље 2) овај проблем решавају на различите начине, али ретко комбинују ширу језичку покривеност, конкретне и одмах применљиве предлоге исправки, и архитектуру која омогућава независно скалирање и отпорност на оптерећење. Алати оријентисани на дубину анализе по правилу захтевају комплекснију припрему (нпр. успешну изградњу пројекта), док алати оријентисани на брзину и једноставност интеграције корисника обично остављају само с листом налаза, без конкретног предлога решења.

1.1 Циљ рада

Циљ овог рада је пројектовање и имплементација платформе RepoOptimizer — дистрибуираног система за статичку анализу изворног кода који подржава више програмских језика кроз заједнички слој парсирања, детектује проблеме из више категорија (code smell обрасци, перформансе, безбедност, дуплиран код), сваком пронађеном проблему додељује оцену озбиљности, и за сваки проблем генерише конкретан предлог исправке прилагођен језику и контексту кода. Посебан циљ је да систем буде пројектован као скуп независних микросервиса, како би се демонстрирали принципи дистрибуираних система — асинхрона комуникација, независно скалирање, отпорност на делимичан пад система — у контексту реалног, функционалног алата, а не искључиво као теоријска вежба.

1.2 Приступ и обим рада

Предложено решење је реализовано као шест микросервиса написаних у програмском језику Rust, повезаних асинхронном разменом порука, уз Веб кориснички интерфејс и CLI алат као два равноправна начина приступа истом систему. Рад обухвата читав развојни циклус: од прегледа постојећих решења и избора технологија, преко пројектовања архитектуре и имплементације појединачних сервиса, до тестирања и демонстрације рада система над конкретним примерима кода. У складу са реалним

обимом изводљивим у оквиру дипломског рада, поједини аспекти система — попут замене рангирања тренираним моделом машинског учења или оркестрације путем Kubernetes-а — нису имплементирани, већ су експлицитно наведени као правци даљег развоја (поглавље 8), уместо да буду прећутани или приказани као део тренутног обима.

1.3 Структура рада

Рад је организован у осам поглавља. Након увода, поглавље 2 даје преглед постојећих решења за статичку анализу кода (SonarQube, Semgrep, CodeQL) и позиционира предложено решење у односу на њих. Поглавље 3 описује коришћене технологије, по слојевима система. Поглавље 4 описује архитектуру система — микросервисе, ток података, модел конкурентности и база података. Поглавље 5 чини највећи део рада и детаљно описује имплементацију сваког микросервиса (Auth, Parser, Analysis, ML Ranker, Suggestion Generator, API Gateway), као и Веб корисничког интерфејса и CLI алата. Поглавље 6 описује приступ тестирању и покривеност јединичним тестовима. Поглавље 7 демонстрира рад система кроз конкретан пример, од пријаве корисника до приказа предлога исправке. Поглавље 8 сумира урађено, истиче предности и ограничења решења, и предлаже правце даљег развоја.

Глава 2

Преглед постојећих решења

Ручна ревизија кода (code review) представља један од најпоузданијих, али истовремено и најспоријих и најскупљих механизма за откривање грешака, безбедносних пропуста и проблема у структури софтвера. Како обим пројеката расте, потреба за аутоматизованим алатима који унапред указују на проблематичне делове кода постаје све израженија. У ту сврху развијен је велики број алата за статичку анализу кода (static code analysis), од којих сваки на другачији начин решава компромис између прецизности, брзине, покривености језика и лакоће интеграције. У наставку су описана три репрезентативна алата која покривају различите приступе проблему, након чега је дат упоредни преглед и позиционирање решења развијеног у оквиру овог рада.

2.1 SonarQube

SonarQube представља једну од најраспрострањенијих платформи за статичку анализу кода, са преко 7000 правила специфичних за појединачне програмске језике, намењених детекцији грешака (bugs), безбедносних рањивости (vulnerabilities) и code smell образаца [1]. Анализа се заснива на парсирању изворног кода у апстрактно синтаксно стабло (AST) и контролно-токовни граф (CFG), над којима се даље примењују правила и рачунају метрике квалитета кода. Поред откривања појединачних проблема, SonarQube прати историјске податке о квалитету кроз време, израчунава технички дуг (technical debt) и интегрише се у CI/CD процесе кроз концепт квалитетних капија (quality gates), чиме се спречава спајање кода који не задовољава дефинисане критеријуме. Платформа покрива преко 30 програмских језика и доступна је како у бесплатној (Community), тако и у комерцијалним варијантама са проширеним могућностима (безбедносне провере, усклађеност са регулативама и др.) [1].

Иако је SonarQube веома зрео и широко усвојен алат, његова архитектура је првенствено замишљена као централизован сервер над којим клијенти покрећу анализу, а проширивање скупом нових правила или језика захтева развој у оквиру SonarSource екосистема.

2.2 Semgrep

Semgrep је лаган (lightweight), open-source алат за статичку анализу који такође парсира изворни код у апстрактно синтаксно стабло, али правила пише у облику који визуелно подсећа на сам изворни код, уместо на формалне AST упите или регуларне изразе [2]. Овакав приступ снижава праг уласка за писање нових правила — програмер без дубљег познавања унутрашње репрезентације кода може описати образац (pattern)

који жели да пронађе, готово истим синтаксним обликом у ком би тај код и сам написао. Semgrep подржава преко 30 језика, ради локално без слања кода на екстерни сервер, и посебно је погодан за интеграцију у CI/CD цевоводе због брзине извршавања [2]. Регистар правила (Semgrep Registry) садржи више хиљада унапред припремљених правила организованих по категоријама (безбедност, исправност, стил).

Semgrep се првенствено ослања на синтаксно препознавање образаца (pattern matching), док је дубља анализа тока података (data-flow, cross-function, cross-file) доступна у оквиру комерцијалне (Pro/AppSec) понуде.

2.3 CodeQL

CodeQL представља семантички механизам анализе кода развијен од стране компаније GitHub, који изворни код не посматра само као текст или синтаксно стабло, већ га преводи у релациону базу података семантичких елемената — класа, метода, променљивих, израза, контролних структура и путања тока података [3]. Над овако добијеном базом могу се извршавати упити писани у декларативном језику QL, концепцијски сличном SQL-у, чиме се омогућава проналажење сложених образаца рањивости, попут праћења корисничког уноса до осетљиве функције без одговарајуће валидације (taint tracking) [3]. За разлику од алата заснованих искључиво на синтаксном поклапању, овакав приступ омогућава откривање рањивости на нивоу логики програма, а не само површинских образаца.

Цена ове прецизности је већа комплексност припреме анализе: за компајлиране језике CodeQL захтева успешну изградњу (build) пројекта како би се база чињеница генерисала, што отежава примену над хетерогеним или непотпуним репозиторијумима.

2.4 Упоредни преглед и позиционирање решења

Табела 1 сумира кључне разлике између описаних алата и платформе RepoOptimizer, развијене у оквиру овог рада.

Табела 1: Упоредни преглед алата за статичку анализу

Критеријум	SonarQube	Semgrep	CodeQL	RepoOptimizer
Приступ анализи	AST + CFG, правила по језику	AST pattern matching	Релациона база + упитни језик QL	AST (Tree-sitter) + скуп правила по категоријама
Вишејезичност	Да (преко 30 језика, засебан модул по језику)	Да (преко 30 језика)	Да, уз захтев за build компајлираних језика	Да — заједнички парсинг слој (Tree-sitter) за све подржане језике
Конкретни предлози исправки	Ограничено (упутства у оквиру правила)	Ограничено	Не (фокус на проналажење, не на исправку)	Да — детерминистички шаблон по типу проблема и језику
Архитектура извршавања	Централизован сервер	CLI / локално извршавање	CLI + GitHub Actions интеграција	Дистрибуирани микросервиси (асинхронна комуникација, паралелна обрада)
Модел проширивања	Захтева рад унутар SonarSource екосистема	Правила у YAML формату, лака за писање	QL упити, стромија крива учења	Rust модули по категорији правила

Из упоредног прегледа произилази да су постојећа решења по правилу оптимизована или за прецизност и дубину анализе (CodeQL), или за једноставност писања и брзину (Semgrep), или за свеобухватно праћење квалитета кроз време (SonarQube), док ретко које комбинује сва три својства са фокусом на давање конкретног, одмах применљивог предлога исправке. RepoOptimizer се позиционира управо на том пресеку: користи заједнички Tree-sitter слој за парсирање како би избегао дуплирање логике по језику, организује анализу кроз независне микросервиси ради хоризонталног скалирања и отпорности на пад појединачне компоненте, и за сваки пронађени проблем генерише конкретан предлог исправке на основу унапред дефинисаних шаблона, уместо да корисника остави само са листом упозорења.

Глава 3

Коришћене технологије

У овом поглављу описане су технологије коришћене приликом развоја платформе RepoOptimizer, груписане према слојевима система: позадински део (backend), обрада и размена података, кориснички интерфејс (frontend) и алати за развој и покретање система (DevOps).

3.1 Rust

Rust је системски програмски језик отвореног кода, првобитно развијен у компанијским Mozilla Research лабораторијама, чији је циљ да програмерима омогући изградњу поузданог и ефикасног софтвера [4]. За разлику од традиционалних системских језика попут C-а и C++-а, Rust гарантује меморијску и нитну (thread) сигурност већ у фази компајлирања, кроз механизам власништва (ownership) и позајмљивања (borrowing) над вредностима, без потребе за garbage collector-ом у време извршавања [4]. Ово својство је било кључно при избору језика за развој микросервиса платформе RepoOptimizer, с обзиром да се ради о систему који паралелно обрађује велики број фајлова из репозиторијума корисника, где грешке у управљању меморијом или тркама нити (data races) могу довести до тешко уочљивих грешака у продукцији. Додатна предност је и перформанса упоредива са C/C++ решењима, уз богат екосистем пакета (crates) доступних преко званичног регистра crates.io.

3.2 Backend технологије

3.2.1 Actix-web и Tokio

Actix-web је веб-framework за Rust намењен изградњи HTTP сервера и REST API-ја, изграђен над асинхроним извршним окружењем (runtime) Tokio [5]. Tokio обезбеђује петљу догађаја (event loop), планирање задатака и асинхроне I/O примитиве које омогућавају да један процес истовремено опслужује велики број конкурентних захтева без блокирања нити извршавања. У оквиру платформе RepoOptimizer, Actix-web се користи као основа свих сервиса који излажу HTTP интерфејс (API Gateway, Auth Service), док се Tokio асинхрони модел користи у свим сервисима који обављају I/O операције (приступ бази, комуникација преко RabbitMQ, мрежни позиви), укључујући и паралелну обраду улазних фајлова приликом анализе репозиторијума (детаљније описано у поглављу 4).

3.2.2 Serde и SQLx

Serde представља стандардни оквир за серијализацију и десеријализацију података у Rust екосистему, коришћен за конверзију интерних структура података у JSON формат приликом комуникације између сервиса (преко RabbitMQ порука) и приликом враћања одговора API Gateway-а ка корисничком интерфејсу. SQLx је асинхрона библиотека за приступ релационим базама података која проверава SQL упите у време компајлирања упоређујући их са стварном шемом базе, чиме се смањује ризик од грешака у типовима података које би се иначе откриле тек у време извршавања. У платформи се користи за комуникацију са PostgreSQL базом из Auth и Analysis сервиса.

3.2.3 MongoDB драјвер

За сервисе којима је потребна флексибилнија, документ-оријентисана шема (Parser сервис — чување AST репрезентације кода; Suggestion Generator сервис — чување генерисаних предлога), користи се званични Rust драјвер за MongoDB. Структура AST-а се разликује од језика до језика и мења се у зависности од граматике коју Tree-sitter користи, па је документ модел погоднији од строго дефинисане релационе шеме.

3.3 Обрада и размена података

3.3.1 Tree-sitter

Tree-sitter је алат за генерисање парсера и библиотека за инкрементално парсирање кода, способна да изгради конкретно синтаксно стабло (concrete syntax tree) изворног фајла и ефикасно га ажурира након локализованих измена, уместо да фајл парсира изнова у целости [6]. Tree-sitter је пројектован да буде довољно општ да парсира широк спектар програмских језика кроз заједнички интерфејс, довољно брз да се користи и при парсирању на сваки притисак тастера у едитору кода, и отпоран на синтаксне грешке у смислу да и даље даје употребљив резултат и када фајл није потпуно валидан [6]. Ова својства чине Tree-sitter погодним избором за Parser сервис платформе RepoOptimizer, где је потребно подржати више програмских језика (Python, JavaScript, Rust, Java, Go) кроз јединствен приступ парсирању, без потребе за развојем посебног парсера за сваки језик.

3.3.2 RabbitMQ

RabbitMQ је посредник за размену порука (message broker) отвореног кода који имплементира протокол AMQP (Advanced Message Queuing Protocol) [7]. У AMQP моделу, произвођачи (publishers) шаљу поруке на размењиваче (exchanges), који на основу правила повезивања (bindings) прослеђују копије порука одговарајућим редовима (queues), одакле их преузимају потрошачи (consumers) [7]. Овакав модел омогућава лабаво повезивање (loose coupling) између произвођача и потрошача порука — сервис који шаље поруку не мора познавати сервис који је обрађује нити чекати да обрада буде завршена. У платформи RepoOptimizer, RabbitMQ представља централни меха-

низам комуникације између свих микросервиса: када корисник покрене анализу, API Gateway креира посао (job) и поставља га у ред, одакле га преузима Parser сервис, а резултат прослеђује даље кроз ланац сервиса (Analysis → ML Ranker → Suggestion Generator) све до агрегације одговора на API Gateway-у. Овај приступ обезбеђује да привремени пад или успорење једног сервиса не обори цео систем, као и могућност независног хоризонталног скалирања сваког сервиса према оптерећењу.

3.3.3 Redis

Redis је меморијска (in-memory) база података типа кључ–вредност. У платформи је њена потврђена употреба на страни API Gateway-а (поглавље 5.6.6), који комплетне, агрегиране резултате завршеног посла анализе кешира под кључем облика `results:: {analysis_job_id}` са роком истека од 3600 секунди (1 час) — чиме се поновљени преглед истих резултата (нпр. корисник освежава страницу са резултатима) опслужује без поновног упита над MongoDB колекцијама. Приступ Redisу је имплементиран као “best-effort”: уколико Redis привремено није доступан, захтев се и даље успешно опслужује директно из MongoDB базе, без грешке видљиве кориснику. Redis интеграција је предвиђена и за ML Ranker сервис (поглавље 5.4.7), ради кеширања израчунатих оцена рангирања.

3.4 Frontend технологије

Кориснички интерфејс платформе развијен је као једнострани апликација (SPA) коришћењем библиотеке React [8] (верзија 19), уз Vite [9] као алат за развој и изградњу (build tool), Tailwind CSS за стилизовање компоненти и react-router-dom (верзија 7) за рутирање на клијентској страни. За приказ и уређивање изворног кода, као и за приказ разлике (diff) између оригиналног и предложеног кода, користи се Monaco Editor [10] — иста едитор компонента на којој је заснован Visual Studio Code — кроз @monaco-editor/react омотач (wrapper). За графички приказ расподеле проблема по озбиљности користи се библиотека recharts, а lucide-react за иконице у корисничком интерфејсу. Комуникација са API Gateway-ом остварена је преко HTTP клијента Axios. Кориснички интерфејс покрива ток регистрације и пријаве корисника, покретање анализе репозиторијума и преглед резултата кроз Dashboard, Login, Register и Result странице (детаљније описано у поглављу 5.7).

3.5 DevOps и квалитет кода

3.5.1 Организација пројекта и контејнеризација

Backend је организован као јединствен Cargo workspace, у којем се сваки микросервис налази у сопственом потпројекту унутар директоријума `services/` (нпр. `services/auth_service`, `services/parser_service`), док CLI алат (поглавље 5.8) чини посебан пакет `workspace-a`, `cli/`, ван овог директоријума. Оваква организација омогућава

дељење заједничких зависности и конфигурације на нивоу workspace-a, уз задржавање јасне границе између сервиса.

За пет од шест сервиса користи се заједнички `Dockerfile.service`, параметризован преко `build` аргумента `SERVICE_NAME`. `Analysis` сервис има посебан `Dockerfile.analysis_service`, с обзиром да позива `Semgrep` као екстерни процес (поглавље 4.3), па његов `runtime image`, за разлику од осталих, мора да садржи `Python` окружење са инсталираним `Semgrep CLI` алатом. Оба `Dockerfile`-а користе двофазни (`multi-stage`) `build`: у првој фази (`builder`, заснованој на званичном `rust image-u`) се прво копирају само `Cargo.toml/Cargo.lock` фајлови свих чланова workspace-a уз привремене празне (`fn main() {}`) верзије `main.rs` фајлова, како би се изградиле и кеширале зависности независно од изворног кода; тек након тога се копира стварни изворни код сервиса и покреће поновна изградња, која због `Docker layer` кеширања поново компајлира само код тог сервиса, а не и зависности. Друга фаза користи минималан `debian:bookworm-slim` image у који се копира само изграђени бинарни фајл, чиме се значајно смањује величина финалног image-a у односу на image који би садржао цео `Rust toolchain`.

3.5.2 Docker Bake и управљање ресурсима при build-u

Како паралелна изградња свих шест сервиса (`Cargo --release build` је меморијски захтеван због зависности попут `sqlx-macros`, `MongoDB` драјвера и нативне компилације `Tree-sitter` граматика) на развојним машинама са ограниченом количином `RAM`-а може довести до прекида процеса услед недостатка меморије, `build` процес је додатно ограничен на два нивоа: број паралелних `rustc` процеса по сервису је ограничен на 2 (`cargo build --jobs 2`), а број сервиса који се граде паралелно контролисан је кроз `docker-bake.hcl` конфигурацију (`Docker Bake`), где променљива `MAX_PARALLEL` унапред ограничава изградњу на један сервис у исто време. За свакодневни развој, препоручени начин рада је покретање појединачних сервиса локално (`cargo run -p <ime_servisa>`) уз инфраструктурне компоненте (базе, `RabbitMQ`) у `Dockeru`, док се `docker compose up` користи за интеграционо тестирање целог система и финалну демонстрацију, како би платформа могла да се покрене идентично на било којој машини са инсталираним само `Dockerom`.

3.5.3 Оркестрација сервиса

`Docker Compose` дефинише шест инфраструктурних и шест апликативних контејнера (табела 2), повезаних преко заједничке `Docker` мреже. Сваки сервис има дефинисан здравствени тест (`healthcheck`) — инфраструктурне компоненте проверавају се алатима специфичним за сваку базу (`pg_isready`, `mongosh --eval`, `redis-cli ping`, `rabbitmq-diagnostics ping`), док се микросервиси проверавају преко сопствене / `health` HTTP руте. Редослед покретања контролисан је преко `depends_on` услова

`service_healthy`, чиме се гарантује да сервис не покуша да успостави конекцију ка бази или ка RabbitMQ-у пре него што је та компонента заиста спремна за рад.

Табела 2: Портови сервиса (docker-compose)

Сервис	Хост порт	Зависи од
PostgreSQL	5433 → 5432	—
MongoDB	27017	—
Redis	6379	—
RabbitMQ (AMQP / management UI)	5672 / 15672	—
Auth Service	8001	PostgreSQL
Parser Service	8002	MongoDB, RabbitMQ
Analysis Service	8003	MongoDB, RabbitMQ
ML Ranker Service	8004	MongoDB, RabbitMQ
Suggestion Generator Service	8005	MongoDB, RabbitMQ, PostgreSQL
API Gateway	8006	MongoDB, RabbitMQ, PostgreSQL, Auth Service

3.5.4 Квалитет кода

Квалитет Rust кода проверен је кроз уграђени тест framework језика, са 62 јединична теста која покривају кључну пословну логику сервиса (детаљније у поглављу 6), док је документација кода генерисана алатом `cargo doc`.

Kubernetes оркестрација и rate limiting на нивоу API Gateway-а разматрани су као могући правци проширења система у продукционом окружењу, али нису део тренутне имплементације — ова одлука је детаљније образложена у поглављу 8 (Закључак).

Глава 4

Архитектура система

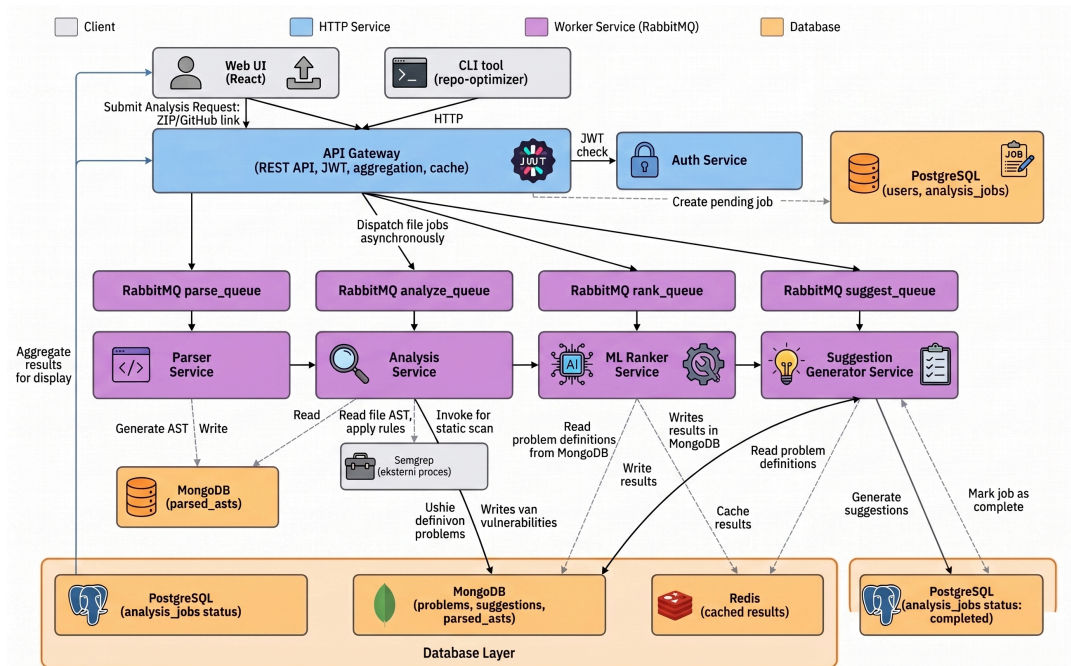
4.1 Преглед архитектуре

RepoOptimizer је пројектован као дистрибуиран систем заснован на микросервисној архитектури, у којој је свака функционална целина (аутентикација, парсирање кода, статичка анализа, рангирање проблема, генерисање предлога) издвојена у засебан, независно развијан и покретан сервис. Сервиси међусобно комуницирају искључиво асинхроно, преко RabbitMQ посредника за размену порука (описано у поглављу 3.3.2), док је API Gateway једина компонента која излаже синхрони REST интерфејс ка корисничком интерфејсу.

Овакав приступ је изабран из неколико разлога:

- **Независно скалирање** — сервиси који представљају уско грло (нпр. Parser сервис при анализи великих репозиторијума) могу се скалирати независно од остатка система, без потребе за умножавањем целокупне апликације.
- **Отпорност на делимичан пад система** — асинхрона комуникација преко редова порука значи да привремена недоступност једног сервиса не прекида рад остатка система; поруке остају у реду до тренутка када сервис поново постане доступан.
- **Независан развој и тестирање** — сваки сервис има јасно дефинисану одговорност и сопствену базу података (или кеш), што омогућава изоловано тестирање без подизања целог система.

Слика 1 приказује преглед архитектуре система и редослед којим захтев корисника пролази кроз компоненте.



Слика 1: Преглед архїїекїїуре: клијенти, API Gateway, RabbitMQ pipeline и базе података

Редослед обраде једног захтева је следећи: корисник кроз Web UI подноси захтев за анализу репозиторијума (отпремљене ZIP архиве или унос GitHub линка); захтев стиже до API Gateway-а, који га аутентичује (JWT) и у PostgreSQL бази креира запис посла (analysis_jobs) са статусом pending, након чега се за сваки фајл репозиторијума шаље засебна порука у RabbitMQ; Parser сервис преузима поруке из реда уз ограничење конкурентности описано у поглављу 4.4, генерише AST за сваки фајл и уписује га у MongoDB; Analysis сервис над AST-ом примењује скуп правила и уписује пронађене проблеме у MongoDB; ML Ranker сервис додељује тежину (severity) сваком проблему, уписује израчунату вредност назад у исти запис у MongoDB и кешира резултат у Redisу ради бржег поновног приступа; Suggestion Generator сервис за сваки проблем генерише предлог исправке на основу шаблона, уписује га у MongoDB и означава посао као завршен (status = completed) у PostgreSQL бази; на крају, API Gateway агрегира резултате из MongoDB колекција и враћа их корисничком интерфејсу.

4.2 Микросервиси

Табела 3 даје преглед микросервиса, њихове одговорности, коришћених технологија и база података које користе.

Табела 3: Преглед микросервиса платформе RepoOptimizer

Сервис	Одговорност	Технологије	База података
Auth Service	Регистрација, пријава, издавање JWT токена	Actix-web, JWT, bcrypt	PostgreSQL (users)
Parser Service	Парсирање изворног кода у AST за више језика	Tree-sitter, Tokio	MongoDB (source_code, parsed_ast)
Analysis Service	Статичка анализа и детекција проблема над AST-ом	Сопствена правила, Semgrep (екстерни процес), Tokio	MongoDB (problems)
ML Ranker Service	Рангирање проблема по тежини (severity)	Rule-based baseline	MongoDB (ажурирање problems) + Redis (кеш)
Suggestion Generator Service	Генерисање предлога исправки	Template engine	MongoDB (suggestions) + PostgreSQL (статус посла)
API Gateway	REST API, аутентикација, агрегација одговора, праћење историје анализа	Actix-web, JWT middleware	PostgreSQL (analysis_jobs), MongoDB (читање problems/suggestions), Redis (кеш резултата)

Сви сервиси, изузев API Gateway-а, комуницирају искључиво преко RabbitMQ-а и не излажу директно доступан мрежни интерфејс кориснику. Расподела база података по сервисима детаљније је образложена у поглављу 4.6.

4.3 Analysis сервис — категорије правила

Analysis сервис примењује скуп правила организованих у четири категорије:

- **Code smell обрасци** — предугачке методе, велике класе, дубоко угњеждавање контролних структура, дуплирани блокови кода;
- **Перформансе** — неефикасне петље, сумњива временска сложеност алгоритама, непотребно копирање/алокације;

- **Безбедност** — SQL инјекције, hard-coded креденцијали, недостатак валидације улазних података;
- **Стил и добре праксе** — конвенције именовања, основна комплексност функција, недостатак документације.

Скуп правила имплементиран је за све подржане језике (Python, JavaScript, Rust, Java, Go). Semgrep се не користи као уграђена библиотека, већ се покреће као екстерни процес (`semgrep scan --json`) из Analysis сервиса, чији се JSON излаз парсира и спаја са налазима интерних детектора — због тога runtime Docker image овог сервиса, за разлику од осталих пет, садржи и Python окружење са инсталираним Semgrep CLI алатом (детаљније у поглављу 3.5). Тамо где се резултати интерних детектора преклапају са резултатима алата Semgrep, примењује се корак дедупликације, тако да корисник не добија дуплиране налазе за исти проблем.

4.4 Конкурентна обрада фајлова

Када корисник поднесе репозиторијум на анализу путем ZIP архиве или GitHub линка, сваки фајл из репозиторијума шаље се као засебна порука у RabbitMQ ред (`parse_queue`), а не као део унапред формираног пакета. Паралелизам се постиже на нивоу сваког сервиса који поруке преузима: сервис ограничава број фајлова који се **истовремено** обрађују помоћу бројачког семафора (`tokio::sync::Semaphore`) са 20 дозвола (`permits`) — при преузимању нове поруке из реда, сервис прво мора да добије слободну дозволу семафора, обради поруку унутар независног асинхроног Tokio задатка (`tokio::spawn`), и по завршетку дозволу враћа. На овај начин је у сваком тренутку највише 20 фајлова у обради по сервису, док се остатак чека у RabbitMQ реду, уместо да се сви фајлови обрађују строго секвенцијално један по један. Овај механизам је посебно битан имајући у виду да број фајлова у реалним репозиторијумима може бити велики, а да се појединачно парсирање и анализа једног фајла могу извршити независно од осталих.

Ограничење на 20 конкурентних задатака представља компромис између брзине обраде и потрошње ресурса (меморија, број отворених конекција ка бази) — потврђено је у Parser сервису (поглавље 5.2.7), док је за остале сервисе који на сличан начин преузимају поруке из RabbitMQ-а (Analysis, ML Ranker, Suggestion Generator) претпостављено да прате исти образац, до потврде када њихов изворни код буде доступан.

4.5 Ток података кроз систем

Редослед обраде једног захтева, корак по корак, дат је у наставку:

1. Web UI → API Gateway: аутентикација корисника (JWT).
2. API Gateway → PostgreSQL: креирање записа посла у табели `analysis_jobs` (статус `pending`).
3. API Gateway → RabbitMQ: слање засебне поруке по фајлу у ред `parse_queue`.

4. RabbitMQ → Parser Service: преузимање порука уз ограничење на 20 конкурентних задатака, генерисање AST-a (упис у MongoDB), прослеђивање у ред `analyze_queue`.
5. Parser Service → Analysis Service: примена правила над AST-ом (упис у MongoDB), прослеђивање у ред `ml_ranker_queue`.
6. RabbitMQ → ML Ranker Service: рангирање проблема по тежини (ажурирање MongoDB записа, кеш у Redisu), прослеђивање у ред `suggestion_generator_queue`.
7. RabbitMQ → Suggestion Generator Service: генерисање предлога исправки (упис у MongoDB); по завршетку, ажурирање статуса посла на `completed` у PostgreSQL табели `analysis_jobs`.
8. API Gateway: агрегација резултата из MongoDB колекција.
9. API Gateway → Web UI: приказ резултата кориснику.

4.6 Праћење статуса послова и историја анализа

За праћење статуса посла анализе и историје анализа по кориснику, систем користи табелу `analysis_jobs` у PostgreSQL бази, а не посебну колекцију у MongoDB. Табела садржи страну везу (foreign key) ка табели `users`, поља `status` (`pending`, `processing`, `completed`, `failed`), `total_files/processed_files` за праћење напретка, као и `created_at/completed_at` временске ознаке.

Ова одлука о распореду података донета је из два разлога:

- **Власништво над послом.** Релациона веза према `users` табели омогућава проверу да ли уловљени корисник заиста сме да приступи резултатима траженог посла, као и листање свих претходних анализа које је корисник покренуо (функционалност “моје анализе” у корисничком интерфејсу).
- **Раздвајање структурираних и неструктурираних података.** Статус и власништво посла су подаци фиксног, релационог облика, док су AST, пронађени проблеми и предлози исправки променљивог облика који зависи од језика и типа проблема — за такве податке MongoDB документ модел је погоднији. Ранија варијанта система је чувала статус посла у засебној MongoDB колекцији, али је то напуштено у корист описаног приступа.

Из истог разлога, систем намерно не чува копију пронађених проблема (`problems`) у PostgreSQL бази — с обзиром да комплетан запис проблема (тежина, линије кода, `rank_score`, и др.) већ постоји у MongoDB колекцији, чување исте информације на два места би захтевало двоструки упис при сваком налазу и уводило ризик од десинхронизације података, без стварне користи.

4.7 Безбедносни аспекти архитектуре

У тренутној имплементацији, безбедност система ослања се на три механизма: JWT токене за аутентикацију корисника на нивоу API Gateway-a; параметризоване упите ка бази података ради спречавања SQL инјекција унутар самог система (не у анали-

зираном коду, већ у коду платформе); и чување креденцијала и конфигурације кроз `environment` варијабле, ван изворног кода. Напредне теме попут HTTPS терминације и енкрипције података у мировању нису део тренутне имплементације, с обзиром да систем у овој фази представља демонстрациони прототип намењен локалном покретању, а не продукционо окружење изложено јавном интернету. Систем не тврди пуну, формалну усклађеност са регулативама попут GDPR-а, али имплементира конкретан механизам права на брисање података (*right to erasure*) на захтев корисника, са провером власништва над подацима — детаљније описан у поглављу 5.6.8.

Глава 5

Имплементација

У овом поглављу детаљно је описана имплементација појединачних микросервиса платформе RepoOptimizer, са фокусом на кључне структуре података, ток извршавања и безбедносне механизме сваког сервиса. Сервиси су описани редоследом којим учествују у обради једног захтева корисника (поглавље 4.5), почевши од Auth Service-а.

5.1 Auth Service

5.1.1 Одговорност сервиса

Auth Service је задужен за регистрацију и пријаву корисника, као и за издавање JWT (JSON Веб Токен) токена којима се корисник даље аутентукује при комуникацији са API Gateway-ом. Сервис је намерно пројектован као минимална, самостална целина — не зависи ни од једног другог микросервиса и комуницира искључиво са сопственом PostgreSQL базом података.

5.1.2 Структура пројекта

Сервис је организован кроз три изворна фајла:

```
auth_service/  
├─ Cargo.toml  
└─ src/  
    ├─ main.rs      – HTTP ruter, obrađivači zahteva (handlers),  
konfiguracija  
    ├─ models.rs    – strukture podataka (DTO-ovi, model korisnika, JWT  
klejmovi)  
    └─ jwt.rs       – kreiranje i verifikacija JWT tokena
```

Оваква организација — јасно раздвајање модела података, HTTP слоја и логике везане за токене у засебне модуле — омогућава да се сваки део сервиса независно тестира и по потреби замени (нпр. замена bcrypt алгоритма хешовања другим, без утицаја на остатак сервиса).

5.1.3 Модел података

Табела 4 приказује кључне структуре података дефинисане у `models.rs`.

Табела 4: Структуре података Auth Service-a

Структура	Намена	Кључна поља
User	Репрезентација реда у табели users (мапирање преко sqlx::FromRow)	id: Uuid, email: String, password_hash: String, created_at, updated_at
RegisterRequest	Тело захтева за регистрацију	email: String, password: String
LoginRequest	Тело захтева за пријаву	email: String, password: String
AuthResponse	Одговор сервиса након успешне регистрације/пријаве	user_id: Uuid, email: String, token: String
Claims	Садржај (payload) JWT токена	sub: String (ид корисника), email: String, exp: usize (време истека)

Структура User одговара users табели у PostgreSQL бази: id је типа UUID са подразумеваном вредношћу gen_random_uuid(), email је UNIQUE NOT NULL (и додатно индексиран, idx_users_email), док су created_at/updated_at типа TIMESTAMP. Ради лакшег локалног тестирања и демонстрације, иницијализациона SQL скрипта уноси и једног тест корисника (test@repo-optimizer.local) са унапред израчунатим bcrypt хешом лозинке, чиме се демонстрација система (поглавље 7) може извести без ручне регистрације.

5.1.4 REST API

Сервис излаже три HTTP руте, приказане у табели 5.

Табела 5: Руте Auth Service-a

Метод	Путања	Опис	Аутентикација
GET	/health	Провера доступности сервиса	Не
POST	/register	Регистрација новог корисника	Не
POST	/login	Пријава постојећег корисника	Не

5.1.5 Ток регистрације

Поступак регистрације, имплементиран у функцији register, састоји се од следећих корака: (1) основна валидација формата мејл адресе; (2) провера да ли мејл адреса већ постоји у бази; (3) хеш лозинке алгоритмом bcrypt; (4) упис новог корисника у базу; (5) генерисање JWT токена и враћање одговора кориснику. Листинг 1 приказује део функције који се односи на хеш лозинке и упис корисника у базу.

```
// Hash password
let password_hash = bcrypt::hash(&payload.password, bcrypt::DEFAULT_COST)?;

// Insert user
let row = sqlx::query(
    "INSERT INTO users (email, password_hash) VALUES ($1, $2) RETURNING id,
    email"
)
    .bind(&payload.email)
    .bind(&password_hash)
    .fetch_one(&state.db)
    .await?;
```

Листинг 1: Хеш лозинке и упис корисника (auth_service/src/main.rs)

Провера да ли мејл адреса већ постоји изведена је као посебан упит пре самог уписа (`SELECT id FROM users WHERE email = $1`), а не ослањањем искључиво на `UNIQUE` ограничење колоне и хватање грешке базе — овим приступом се кориснику враћа јасна порука о конфликту (HTTP 409) уместо генеричке грешке базе података.

5.1.6 Ток пријаве

Пријава корисника (функција `login`) прати симетричан поступак: проналажење корисника по мејл адреси, провера лозинке поређењем са сачуваним хешом (`bcrypt::verify`), и генерисање новог JWT токена при успешној провери. Уколико корисник не постоји или лозинка не одговара, сервис у оба случаја враћа исту поруку грешке (“Invalid credentials”) са статусом 401, чиме се спречава да нападач на основу разлике у одговору сервера закључи да ли мејл адреса постоји у систему (user enumeration).

5.1.7 JWT аутентикација

Креирање и верификација токена издвојени су у модул `jwt.rs`, коришћењем библиотеке `jsonwebtoken`. Токен се потписује HMAC алгоритмом (подразумевани `Header` из библиотеке), а садржи идентификатор корисника, мејл адресу и време истека, конфигурисано преко `environment` променљиве `JWT_EXPIRATION_HOURS` (подразумевано 24 часа). Листинг 2 приказује функцију за креирање токена.

```
pub fn create_jwt(user_id: &str, email: &str) -> Result<String, JwtError> {
    let secret = env::var("JWT_SECRET").unwrap_or_else(|_|
"dev_secret".to_string());
    let expiration_hours: i64 = env::var("JWT_EXPIRATION_HOURS")
        .ok().and_then(|h| h.parse().ok()).unwrap_or(24);

    let exp = (chrono::Utc::now() +
chrono::Duration::hours(expiration_hours)).timestamp() as usize;
    let claims = Claims { sub: user_id.to_string(), email:
email.to_string(), exp };

    encode(&Header::default(), &claims,
&EncodingKey::from_secret(secret.as_bytes()))
        .map_err(|_| JwtError::TokenCreation)
}
```

Листинг 2: Креирање JWT токена (auth_service/src/jwt.rs)

Функција за верификацију токена (verify_jwt) дефинисана је у истом модулу и намењена је коришћењу од стране API Gateway-а приликом провере JWT-а на заштићеним рутама (описано у поглављу 5.6).

5.1.8 Безбедносна разматрања и ограничења

У складу са транспарентним приступом опису тренутног стања имплементације који је усвојен у овом раду, потребно је истаћи неколико поједностављења присутних у тренутној верзији Auth Service-а:

- **Подразумевани тајни кључ.** Уколико environment променљива JWT_SECRET није постављена, сервис користи фиксну подразумевану вредност (dev_secret). Ово је прихватљиво за локално развојно окружење, али захтева да JWT_SECRET буде експлицитно постављен у сваком нетривијалном (продукционом) окружењу.
- **Минимална валидација мејл адресе.** Валидност формата мејл адресе проверава се искључиво присуством карактера @, без провере према формалном обрасцу (regex) или слања верификационог мејла.
- **Пермисивна CORS политика.** Сервис користи CorsLayer::permissive(), који дозвољава захтеве са било ког порекла (origin) — погодно за локалну демонстрацију система, али не и за продукционо окружење.
- **Одсуство механизма за обнављање токена (refresh token).** Тренутна имплементација користи искључиво краткорочне JWT токене са фиксним временом истека (подразумевано 24 часа). Након истека, корисник се мора поново пријавити. Табела за чување refresh tokena и одговарајуће /api/auth/refresh и /api/auth/logout руте планиране су као проширење ван оквира овог рада.

Ова ограничења нису последица превида, већ последица опредељења да се у оквиру дипломског рада приоритет да комплетности архитектуре и тока података кроз систем, док су поједина безбедносна појачања сврстана у правце даљег развоја (поглавље 8).

5.2 Parser Service

5.2.1 Одговорност сервиса

Parser Service је задужен за претварање изворног кода појединачног фајла у апстрактно синтаксно стабло (AST) и за израчунавање основних метрика кода (број линија, цикломатска сложеност, дубина угњеждавања, број и величина функција/класа), које даље користе Analysis и ML Ranker сервиси. Сервис не излаже функционалан REST API у смислу пословне логике — HTTP слој своди се на /health руту за проверу доступности — већ посао преузима искључиво асинхроно, као потрошач (consumer) RabbitMQ реда parse_queue.

5.2.2 Структура пројекта

```

parser_service/
├── Cargo.toml
└── src/
    ├── main.rs          – HTTP sloj, inicijalizacija, pokretanje worker-a
    └── models.rs        – Language enum, DTO-ovi, CodeMetrics/
FunctionInfo/ClassInfo
├── rabbitmq.rs         – RabbitMQ konzument, konkurentnost,
prosleđivanje sledećem servisu
└── parsers/
    ├── mod.rs          – LanguageParser trejt, fabrička funkcija,
deljene metrike
    ├── python.rs
    ├── javascript.rs
    ├── rust.rs
    ├── java.rs
    └── go.rs

```

5.2.3 Апстракција LanguageParser

Подршка за више програмских језика решена је кроз трејт LanguageParser, који дефинише заједнички интерфејс за парсирање и извлачење информација, независно од конкретног језика (Листинг 3). Сваки подржани језик има сопствену имплементацију овог трејта (нпр. PythonParser, RustParser), док фабричка функција get_parser на основу вредности Language енум-а бира одговарајућу имплементацију.

```
pub trait LanguageParser {
    fn parse_code(&mut self, code: &str) -> Result<Tree, String>;
    fn extract_functions(&self, tree: &Tree, code: &str) ->
Vec<FunctionInfo>;
    fn extract_classes(&self, tree: &Tree, code: &str) -> Vec<ClassInfo>;
    fn calculate_metrics(&self, tree: &Tree, code: &str) -> CodeMetrics;
}

pub fn get_parser(language: &Language) -> Box<dyn LanguageParser + Send> {
    match language {
        Language::Python => Box::new(python::PythonParser::new()),
        Language::JavaScript =>
Box::new(javascript::JavaScriptParser::new()),
        Language::Rust => Box::new(rust::RustParser::new()),
        Language::Java => Box::new(java::JavaParser::new()),
        Language::Go => Box::new(go::GoParser::new()),
    }
}
```

Листинг 3: Трејт LanguageParser и фабричка функција (parser_service/src/parsers/mod.rs)

Овакав дизајн одговара обрасцу Strategy у комбинацији са Factory Method обрасцем — позивалац (process_and_save функција у main.rs) не мора да познаје конкретан тип парсера, већ ради искључиво над LanguageParser интерфејсом, што новом језику оставља јасну тачку проширења (имплементација трејта и додавање једног реда у get_parser) без измене остатка сервиса.

5.2.4 Парсирање и израчунавање метрика

Свака имплементација LanguageParser-а иницијализује Tree-sitter parser са одговарајућом граматиком (Листинг 4 приказује иницијализацију за Python), након чега парсирање даје стабло (Tree) над којим се даље рекурзивно обилазе чворови.

```

pub struct PythonParser {
    parser: Parser,
}

impl PythonParser {
    pub fn new() -> Self {
        let mut parser = Parser::new();
        parser
            .set_language(tree_sitter_python::language())
            .expect("Failed to load Python grammar");
        Self { parser }
    }
}

```

Листинг 4: Иницијализација Python парсепа (parser_service/src/parsers/python.rs)

Извлачење функција и класа имплементирано је рекурзивним обиласком AST-a, где се сваки чвор проверава према врсти (`node.kind()`), специфичној за граматику датог језика — на пример, дефиниција функције у Pythonу одговара врсти чвора `function_definition`, док исти концепт у Rust граматици одговара врсти `function_item`. За сваку пронађену функцију израчунавају се број параметара, број линија кода, цикломатска сложеност и дубина угњеждавања.

За разлику од извлачења функција и класа, које је специфично за сваки језик, израчунавање цикломатске сложености и дубине угњеждавања имплементирано је као заједничка функција у `parsers/mod.rs` (`calculate_cyclomatic_complexity`, `calculate_nesting_depth`), која обилази AST и препознаје контролне структуре на основу листе врста чворова које покривају све подржане језике (нпр. `if_statement`, `for_statement`, `while_statement`, као и језички специфичне варијанте логичких оператора — `boolean_operator` за Python, `logical_expression` за JavaScript, `lazy_boolean_expression` за Rust). Овим приступом је избегнуто дуплирање исте логице у свакој језичкој имплементацији.

5.2.5 Модел података

Табела 6: Кључне структуре података Parser Service-a

Структура	Намена
Language	Набројиви тип подржаних језика (Python, JavaScript, Rust, Java, Go)
ParseRequest	Улазна порука из RabbitMQ-a — код, језик, <code>analysis_job_id</code> , путања фајла
CodeMetrics	Агрегатне метрике фајла (линије, функције, класе, комплексност, дубина угњеждавања)
FunctionInfo	Подаци о појединачној функцији (назив, опсег линија, број параметара, комплексност)
ClassInfo	Подаци о појединачној класи (назив, опсег линија, број метода)
ParsedAst	Документ који се уписује у MongoDB колекцију <code>parsed_ast</code> — код, језик, метрике, функције, класе

5.2.6 Асинхрона обрада и интеграција са RabbitMQ-ом

Worker који преузима поруке из RabbitMQ реда покреће се у засебном асинхронном Tokio задатку (`tokio::spawn`) приликом старта сервиса, паралелно са HTTP сервером. Конекција ка RabbitMQ-у је обавијена петљом са експоненцијалним одлагањем (`exponential backoff`, почев од 2 секунде до највише 30 секунди) — уколико конекција привремено није доступна (нпр. при паралелном подизању контејнера, када RabbitMQ још није спреман) или се прекине у току рада, worker покушава поновно повезивање уместо да трајно престане са радом, док `/health` HTTP рута остаје независно доступна и не открива овај проблем.

Након успешне обраде фајла, сервис шаље поруку у следећи ред `pipeline`-а (`analyze_queue`) и потврђује пријем оригиналне поруке (`ack`). Уколико обрада не успе (нпр. неподржан језик или неуспешно парсирање), порука се одбија без поновног враћања у ред (`nack` са `queue: false`) — тренутна имплементација нема механизам за поновни покушај нити ред за неиспоручиве поруке (`dead-letter queue`), па се неуспешно обрађени фајлови у овом тренутку једноставно прескачу.

5.2.7 Ограничавање конкурентности

Број фајлова који се истовремено обрађују ограничен је бројачким семафором са 20 дозвола, иницијализованим једном при покретању worker-a.

Петља конзумента преузима поруку из реда тек пошто затражи и добије слободну дозволу семафора (`acquire_owned`), а обраду препушта новом Tokio задатку — дозвола се аутоматски ослобађа када задатак и његов `_permit` изађу из опсега (`scope`). Тиме је у сваком тренутку највише 20 фајлова истовремено у обради унутар овог сервиса,

док остале поруке чекају у RabbitMQ реду, чиме се спречава неограничен раст броја паралелних задатака (и последично меморије) када корисник поднесе репозиторијум са великим бројем фајлова. Исти образац је потврђен и у преостала три worker сервиса (Analysis — поглавље 5.3.10, ML Ranker — поглавље 5.4.7, Suggestion Generator — поглавље 5.5.8).

5.3 Analysis Service

5.3.1 Одговорност сервиса

Analysis Service је централна компонента статичке анализе — над AST-ом и метрикама које је израчунао Parser Service, примењује скуп интерних детектора организованих по категоријама из поглавља 4.3 (code smell, перформансе, безбедност, дуплирани код), комбинује их са налазима алата Semgrep, уклања дубликате између та два извора и уписује коначну листу проблема у MongoDB колекцију problems. Као и Parser Service, посао преузима искључиво асинхроно, као потрошач RabbitMQ реда analyze_queue.

5.3.2 Структура пројекта

```
analysis_service/
├─ Cargo.toml
├─ src/
│   └─ main.rs           ─ HTTP sloj, orkestracija analize jednog
fajla
│   └─ models.rs         ─ Problem, ProblemType, Severity, DTO-ovi
│   └─ rabbitmq.rs       ─ RabbitMQ konzument (analyze_queue →
rank_queue)
│   └─ semgrep.rs        ─ integracija sa eksternim Semgrep procesom
│   └─ dedup.rs          ─ deduplikacija nalaza internih detektora i
Semgrep-a
│   └─ detectors/
│       └─ mod.rs         ─ trejt Detector
│       └─ smells.rs      ─ code smell pravila
│       └─ performance.rs ─ pravila za performanse
│       └─ security.rs    ─ bezbednosna pravila
│       └─ duplication.rs ─ detekcija dupliranog koda
```

5.3.3 Detector апстракција

Слично решењу у Parser сервису (поглавље 5.2.3), сваки детектор имплементира заједнички трејт Detector:

```
pub trait Detector {
    fn detect(&self, data: &crate::models::ParsedAst) -> Vec<Problem>;
}
```

Листинг 5: Трејт Detector (analysis_service/src/detectors/mod.rs)

Функција `process_analysis` у `main.rs` редом позива све имплементације (`SmellDetector`, `PerformanceDetector`, `SecurityDetector`, `DuplicationDetector`), сакупља њихове налазе, додаје им резултат `Semgrep` скенирања, примењује дедупликацију (поглавље 5.3.7) и тек онда уписује коначну листу у базу — исти образац (`Strategy`) као код језичких парсера, само примењен на правила уместо на језике.

5.3.4 Детектори засновани на метрикама кода (code smell)

`SmellDetector` ради искључиво над метрикама које је већ израчунао `Parser Service` (број линија, број параметара, дубина угњеждавања, цикломатска сложеност), без поновног парсирања кода. Правила и њихови прагови приказани су у табели 7.

Табела 7: Прагови за code smell проблеме

Проблем	Услов окидања	Severity (основно / повишено)
Long Method	функција дужа од 50 линија	Medium / High (> 100 линија)
Long Parameter List	више од 5 параметара	Medium / High (> 7 параметара)
Deep Nesting	дубина угњеждавања > 4	Medium / High (> 6)
Complex Method	цикломатска сложеност > 10	Medium / High (> 20)
Large Class	> 500 линија или > 20 метода	Medium / High (> 1000 линија)
Long File	> 500 линија кода у фајлу	Medium / High (> 1000 линија)
Magic Number	бројевни литерал са 2+ цифре (или било који децимални број) ван стринг литерала, коментара и декларације <code>SCREAMING_CASE</code> константе	Low

За детекцију магичних бројева примењена је додатна хеуристика како би се избегли очигледни лажно позитивни налази: једноцифрени бројеви (укључујући уобичајене индексе и кораке петље попут 0, 1, 2) се не пријављују, бројеви унутар стринг литерала се претходно уклањају из линије пре провере, а линије које саме представљају декларацију именоване константе (нпр. `MAX_RETRIES = 42`) се прескачу, с обзиром да је вредност већ именована.

5.3.5 Детектори перформанси

`PerformanceDetector` пријављује три врсте проблема: дубоко угњеждене петље високе сложености (цикломатска сложеност > 15 и дубина угњеждавања > 3), потенцијалне $N+1$ упите над базом података унутар тела петље, и конкатенацију стрингова оператором `+=` унутар петље ($O(n^2)$ понашање).

Детекција H+1 упита заслужује посебан осврт, јер је имплементација свесно прошла кроз итерацију након ученог проблема у ранијој верзији. Првобитни, наивни приступ је проверавао да ли цео фајл истовремено садржи реч `for` и реч попут `query/select`, без икакве везе између њихових позиција — такав приступ је давао лажне позитивне налазе (нпр. SQL упит дефинисан ван било које петље) и, када не би пронашао линију са обе речи истовремено, налаз би погрешно пријављивао на линији 1 као подразумевану вредност. Тренутна имплементација (Листинг 6) уместо тога проверава да ли се позив налик на упит базе података (`execute()`, `.query()`, `.filter()`, `select`, `.objects` и сл.) заиста налази унутар тела петље, где се тело одређује по увучености линија (`indentation`) у односу на линију са `for/while`.

```
let is_loop_start = lower.starts_with("for ") || lower.starts_with("for(")
    || lower.starts_with("while ") || lower.starts_with("while(");
if !is_loop_start { continue; }

for (j, body_line) in lines.iter().enumerate().skip(i + 1) {
    if body_line.trim().is_empty() { continue; }
    let body_indent = body_line.len() - body_line.trim_start().len();
    if body_indent <= indent { break; } // izašli iz tela petlje

    if query_patterns.iter().any(|p| body_line.to_lowercase().contains(p))
    {
        // prijavi na STVARNOJ liniji poziva, ne na liniji 1
        break;
    }
}
```

Листинг 6: Детекција H+1 упита по увучености тела петље, скраћено (`analysis_service/src/detectors/performance.rs`)

Овај приступ је довољно тачан за Python (где је увученост део синтаксе) и представља довољно добру апроксимацију за језике C-стила, с обзиром да је код у пракси готово увек доследно увучен. Регресиони тест (`no_false_positive_when_select_is_outside_any_loop`) експлицитно проверава да SQL упит дефинисан ван петље више не генерише лажан налаз.

Детекција конкатенације стрингова у петљи примењује сличну логику претраге тела петље, уз регуларни израз који искључује уобичајене нумеричке акумулаторе (`total +=`, `count +=` и сл.), како се не би лажно пријавило сабирање бројева као проблем конкатенације стрингова.

5.3.6 Детектори безбедности

`SecurityDetector` је у потпуности заснован на регуларним изразима над изворним кодом (линија по линија), без ослањања на AST, и покрива три врсте проблема:

- **Хардкодоване тајне** — обрасци за API кључеве, лозинке и AWS креденцијале. Regex за AWS креденцијале је проширен током развоја након уоченог пропуста: ранији образац је тражио дословно `aws_secret` непосредно испред знака `=`, чиме није препознавао уобичајен назив променљиве `aws_secret_key` (где се између `aws_secret` и `=` налази додатни део имена `_key`). Исправљен образац препознаје произвољну комбинацију префикса `aws` и кључних речи (`access`, `secret`, `key`, `token`) у имену променљиве, уз одговарајући регресиони тест.
- **XSS рањивости** — искључиво за JavaScript фајлове, препознавањем образаца `innerHTML =`, `document.write()` и `eval()`.
- **Небезбедно генерисање случајних вредности** — употреба Python `random` модула или JavaScript `Math.random()` у контексту који указује на безбедносну намену. Озбиљност налаза зависи од контекста: уколико се име променљиве или околни код односи на појмове попут `token`, `secret`, `session_id`, `csrf` или `otp`, налаз добија `severity High` (уз предлог криптографски сигурне алтернативе — `secrets` модула у Pythonу, односно `crypto.getRandomValues()` у JavaScript-у); у супротном се пријављује као `Low`, с обзиром да је употреба у небезбедном контексту (нпр. бирање насумичне боје) легитимна.

Детекција SQL инјекције, иако је дефинисана као вредност у `ProblemType` еnum-у (поглавље 5.3.7), **тренутно нема сопствену имплементацију** у оквиру `SecurityDetector`-а — покривена је искључиво кроз `Semgrep-ov p/default` скуп правила. Ово је транспарентно наведено као познато ограничење тренутне имплементације, а не превид: писање поузданог детектора SQL инјекције без лажних позитивних налаза (који разликује параметризоване упите од конкатенације стрингова) захтева дубљу анализу тока података него што је regex-based приступ у стању да пружи, па је одлука донета да се овај случај препусти Semgrep-у, чији скуп правила већ покрива ову категорију за све подржане језике осим за Јаву (поглавље 2.4).

5.3.7 Детекција дуплираног кода

`DuplicationDetector` препознаје дословно (Type-1) дуплиране блокове кода — идентичан низ од најмање 6 узастопних линија који се понавља на два или више места у истом фајлу. Алгоритам ради по принципу клизног прозора (`sliding window`): за сваку позицију у фајлу се узима прозор од 6 линија, нормализује се (уклањањем водеће/пратеће празнине) и бележи у хеш мапу виђених блокова. Уколико се идентичан нормализован блок већ појавио раније у фајлу, пријављује се дупликат, који се затим похлепно (`greedy`) проширује линију по линију док се одговарајуће линије и даље поклапају, како би се цео дуплиран сегмент пријавио као један налаз уместо као низ преклапајућих ситних налаза. Блокови са премало не-празних линија или премало укупних карактера (`< 40`) се прескачу, чиме се избегавају лажни налази над понављајућим тривијалним линијама попут самосталних затворених заграда.

5.3.8 Интеграција са Semgrep-ом

Позив ка Semgrep-у имплементиран је у модулу `semgrep.rs`: код фајла се уписује у привремени фајл са екстензијом која одговара језику (`.py`, `.js`, `.ts`, `.rs`, `.java`), након чега се покреће екстерни процес `semgrep scan --json --config=p/default --metrics=off --disable-version-check <putanja>` (Листинг 7). Излаз у JSON формату се парсира, а Semgrep-ова сопствена класификација озбиљности (`ERROR/WARNING/INFO`) мапира се на интерну Severity скалу (`Critical/Medium/Low`). Уколико Semgrep у оквиру налаза врати и предлог аутоматске исправке (поље `extra.fix`), тај предлог се директно преузима у поље `autofix Problem` структуре, независно од шаблонског механизма Suggestion Generator сервиса.

```
let output = Command::new("semgrep")
    .arg("scan")
    .arg("--json")
    .arg("--config=p/default")
    .arg("--metrics=off")
    .arg("--disable-version-check")
    .arg(temp_path)
    .output()
    .await
    .map_err(|e| format!("Failed to execute semgrep. Is it installed on the
system? Error: {}", e))?;
```

Листинг 7: Позив Semgrep-а као екстерног процеса (`analysis_service/src/semgrep.rs`)

Коришћење унапред дефинисаног скупа правила `p/default` уместо Semgrep-ове `auto` опције (која би покушала да преузме додатна правила и пошаље телеметрију са сервиса) обезбеђује предвидљиво, репродуктивно понашање без зависности од мрежне доступности спољних Semgrep сервиса током анализе.

5.3.9 Дедупликација налаза

С обзиром да исти проблем у коду могу пријавити и интерни детектор и Semgrep (нпр. хардкодована лозинка), модул `dedup.rs` (Листинг 8) пре уписа у базу уклања такве дубликате. Сваки налаз се сврстава у грубу категорију (`bucket`) на основу кључних речи у типу проблема (`secret`, `sql_injection`, `xss`, `long_complex`, `insecure_random`, `string_concat_loop`, `loop_nesting`, `unoptimized_query`, `other`) — иста логика категоризације се, намерно, понавља и у Suggestion Generator сервису (поглавље 5.5), како би се “исти проблем” доследно препознао у оба сервиса независно од тога да ли га је пронашао интерни детектор или Semgrep. Два налаза се сматрају дубликатом уколико потичу из истог фајла, припадају истој категорији и њихови опсежи линија се преклапају. Када се открије дубликат, приоритет увек има налаз интерног детектора (чији су `problem_type`, `severity` и порука боље усклађени са остатком `pipeline-a`), док се Semgrep налаз одбацује.

```
if same_file && same_bucket && overlapping && candidate_bucket != "other" {
    let existing_is_custom =
is_custom_detector_finding(&existing.problem_type);
    if !existing_is_custom && candidate_is_custom {
        *existing = candidate; // interni detektor ima prioritet nad
Semgrep nalazom
    }
    continue 'outer; // u svakom slučaju, duplikat se ne dodaje kao novi
unos
}
```

Листинг 8: Правило приоритета при дедупликацији, скраћено (analysis_service/src/dedup.rs)

Категорија `other` је намерно изузета из спајања — налази који не припадају ниједној препознатој категорији се никада не третирају као дупликати једни других, како би се избегло погрешно спајање суштински различитих проблема у истој области кода.

У пракси, стварно преклапање између интерних детектора и Semgrep-а ограничено је претежно на безбедносну категорију (хардкодоване тајне, XSS, insecure random) — Semgrep-ov `p/default` скуп правила је првенствено безбедносно оријентисан и не садржи правила заснована на метрикама кода (број линија, цикломатска сложеност, дубина угњеждавања) нити хеуристику за $N+1$ упите или детекцију дуплираног кода специфичну за овај пројекат, па детектори из поглавља 5.3.4, 5.3.5 и 5.3.7 у пракси немају Semgrep парњака и ретко када бивају уклоњени механизмом дедупликације.

5.3.10 Поузданост конекција

Конекција ка MongoDB бази конфигурирана је са експлицитним параметрима отпорности: максимално 200 и минимално 10 конекција у `connection pool`-у, тајмаут повезивања и селекције сервера од 10 секунди, као и аутоматско понављање неуспешних уписа и читања (`retry_writes`, `retry_reads`). При покретању сервиса креира се и индекс над пољем `analysis_job_id` у колекцији `parsed_ast`, ради бржег проналажења AST-а приликом обраде сваког посла. RabbitMQ конзумент прати идентичан образац отпорности и ограничавања конкурентности описан за Parser Service (поглавље 5.2.6–5.2.7): петља са експоненцијалним одлагањем за поновно повезивање и семафор са 20 дозвола који ограничава број фајлова који се истовремено анализирају, са преузимањем порука из `analyze_queue` и прослеђивањем у `rank_queue`.

5.4 ML Ranker Service

5.4.1 Одговорност сервиса

ML Ranker Service преузима проблеме које је пронашао Analysis Service и сваком додељује нумеричку оцену озбиљности (`rank_score`, у опсегу 0–1), на основу које се проблеми у корисничком интерфејсу приказују ранжирани од најкритичнијег ка

најмање битном. Као што је наведено у поглављу 3.1.4 и README документацији пројекта, овај сервис тренутно представља детерминистички, ручно калибрисан модел рангирања (rule-based baseline), а не тренирани машински модел — именовање сервиса и појединих структура (feature_importance, ProblemFeatures) свесно прате терминологију машинског учења, с обзиром да је замена овог модела стварно тренираним моделом (нпр. XGBoost) најављена као правац даљег развоја (поглавље 8), при чему би улазна обележја описана у поглављу 5.4.3 могла послужити као полазни скуп обележја (feature set) за такав модел.

5.4.2 Структура пројекта

```
ml_ranker_service/  
├─ Cargo.toml  
└─ src/  
    ├─ main.rs      – HTTP sloj, orkestracija rangiranja jednog problema  
    ├─ models.rs    – ProblemFeatures, RankedProblem, DTO-ovi  
    ├─ ml_engine.rs – izračunavanje obeležja i formula rangiranja  
    └─ rabbitmq.rs  – RabbitMQ konzument (rank_queue → suggest_queue)
```

5.4.3 Обележја коришћена за рангирање

За сваки проблем се, у структури ProblemFeatures, израчунава седам улазних обележја, приказаних у табели 8.

Табела 8: Обележја коришћена при рангирању проблема

Обележје	Опис	Начин израчунавања
severity_base	Нормализована основна озбиљност коју је доделио Analysis Service	<code>severity.to_score()</code> / 100.0
type_weight	Тежина категорије проблема	Табела ручно додељених вредности по типу проблема (поглавље 5.4.4)
is_security	Да ли проблем припада безбедносној категорији	Логичка вредност на основу <code>ProblemType</code>
cyclomatic_normalized	Цикломатска сложеност функције која садржи проблем, нормализована	Проналажење функције чији опсег линија садржи проблем (на основу података из Parser сервиса) / горња граница 50
nesting_normalized	Дубина угњеждавања функције, нормализована	Горња граница 10
position	Релативна позиција проблема у фајлу	$1 - (\text{linija_problema} / \text{ukupno_linija})$ — проблеми ближе врху фајла добијају нешто већу вредност
loc_normalized	Дужина функције у линијама, нормализована	Горња граница 200

Поред наведених, израчунавају се и обележја `params_count_normalized` (број параметара функције) и `file_size_normalized` (величина целог фајла), која се чувају у структури `ProblemFeatures`, али се у тренутној верзији формуле рангирања (поглавље 5.4.5) не користе — задржана су као припремљен, али још неактивиран, део скупа обележја за будуће калибрације.

5.4.4 Тежина категорије проблема

Функција `category_weight` додељује сваком типу проблема фиксну тежину у опсегу $[0, 1]$, на основу припадности категорији и процењене озбиљности те категорије у пракси (табела 9).

Табела 9: Тежине категорија проблема (извод)

Категорија проблема	Тежина
Хардкодоване тајне	1.00
SQL инјекција	0.97
XSS рањивост	0.93
Небезбедно генерисање случајних вредности	0.88
Остали безбедносни проблеми	0.85
N+1 упит / неоптимизован упит	0.78
Конкатенација стрингова у петљи	0.70
Угњеждена петља високе сложености	0.68
Остали проблеми перформанси	0.65
Дуплиран код	0.60
Дуга / комплексна метода	0.52
Велика класа / дугачак фајл	0.50
Дубоко угњеждавање	0.48
Предугачка листа параметара	0.44
Магични број	0.28
(подразумевано, непознат тип)	0.40

Овакво рангирање категорија одражава опредељење да безбедносни проблеми (посебно они са директним ризиком по тајност података) по правилу треба да имају приоритет над проблемима читљивости и стила кода, чак и када је њихова основна Severity вредност (поглавље 5.3, табела модела) формално иста.

Додатно, функција `file_exposure_multiplier` увећава израчунату оцену за 25% уколико се проблем налази у фајлу чије име указује на директну изложеност система (нпр. `main`, `server`, `handler`, `controller`, `route`, `api`, `auth`, `gateway`), односно умањује је на 60% уколико се фајл препознаје као помоћни или генерисан код (нпр. `test`, `спес`, `mock`, `migration`, `generated`, `util`) — у осталим случајевима оцена остаје непромењена.

5.4.5 Формула рангирања

Коначна оцена се рачуна као тежинска линеарна комбинација шест од седам обележја из табеле 8 (обележје `position` улази као `1 - position`), након чега се резултат ограничава (`clamp`) на опсег `[0, 1]` и множи фактором изложености фајла (Листинг 9).

```
let raw_score =  
    features.is_security as u8 as f64          * 0.28  
    + features.type_weight                    * 0.22  
    + Self::severity_calibrated(features.severity_base) * 0.20  
    + features.cyclomatic_normalized          * 0.08  
    + features.nesting_normalized            * 0.06  
    + (1.0 - features.position)              * 0.04  
    + features.loc_normalized                * 0.02;  
  
let score = (raw_score.clamp(0.0, 1.0)) *  
file_exposure_multiplier(file_path);  
score.clamp(0.0, 1.0)
```

Листинг 9: Формула рангирања проблема, скраћено (ml_ranker_service/src/ml_engine.rs)

Збир свих седам приказаних коефицијената (укључујући и коефицијент 0.10 декларисан за `file_exposure` у листи `feature_importance` која се враћа кориснику ради објашњивости рангирања) износи тачно 1.00, док се у самом израчунавању шест обележја сабира адитивно (укупно 0.90), а допринос изложености фајла се примењује мултипликативно након тога, а не као седма адитивна компонента. Ово је намерна одлука дизајна — изложеност фајла треба да делује као множилац постојеће оцене (подиже или спушта већ израчунату озбиљност), а не као независан адитивни допринос.

Ради делимичне објашњивости рангирања, сваки `RankedProblem` у одговору сервиса носи и вектор `feature_importance` са декларисаним тежинама из табеле — реч је о истом, глобално фиксном вектору за све проблеме (одражава општу тежину обележја у формули), а не о индивидуалном доприносу обележја за тај конкретан проблем.

5.4.6 Калибрација озбиљности

Функција `severity_calibrated` додатно преобликује сирову, линеарно нормализовану вредност озбиљности (`severity_base`) у калибрисану вредност по комадно-линеарној (piecewise linear) функцији, са циљем да се повећа размак између суседних нивоа озбиљности (`Critical`, `High`, `Medium`, `Low`) и добије равномерније распоређена коначна оцена, уместо да оригиналне вредности (90/70/50/30, видети поглавље 5.3, модел `Severity`) остану груписане близу средине опсега. Калибрисане вредности су тако подешене да сваки ниво озбиљности падне у засебан, међусобно раздвојен опсег: `Critical` ≥ 0.80 , `High` у $[0.58, 0.80)$, `Medium` у $[0.30, 0.58)$ и `Low` у $[0.08, 0.30)$ — исправност овог раздвајања, као и строга монотоност између нивоа, потврђена је тестом `severity_calibration_bands` (поглавље 6.5).

5.4.7 Упис резултата и интеграција

Након израчунавања оцене, сервис ажурира постојећи запис проблема у MongoDB колекцији `problems` (уписује `rank_score` и `feature_importance` у већ постојећи документ, уместо да креира нови), а резултат додатно кешира у Redisу ради бржег поновног приступа при поновљеним упитима над истим послом анализе (поглавље 3.3.3). RabbitMQ конзумент прати исти образац отпорности и ограничавања конкурентности као Parser и Analysis сервис (поглавља 5.2.6–5.2.7, 5.3.10): експоненцијално одлагање при поновном повезивању и семафор са 20 дозвола, са преузимањем порука из `rank_queue` и прослеђивањем у `suggest_queue`.

За разлику од `parse_queue/analyze_queue`, где једна порука носи садржај једног фајла, поруке на `rank_queue` и `suggest_queue` носе листу **свих проблема пронађених у једном фајлу** (резултат рада Analysis сервиса над тим фајлом, након дедупликације), заједно са идентификатором посла (`analysis_job_id`). Семафор са 20 дозвола у ML Ranker и Suggestion Generator сервису тако ограничава број фајлова (тачније, њихових листа проблема) који се истовремено обрађују, на исти начин као и код претходна два сервиса.

5.5 Suggestion Generator Service

5.5.1 Одговорност сервиса

Suggestion Generator Service представља последњу карику pipeline-а: за сваки рангирани проблем генерише конкретан, језички прилагођен предлог исправке, уписује га у MongoDB колекцију `suggestions`, и — с обзиром да је последњи сервис у ланцу — ажурира напредак посла у PostgreSQL табели `analysis_jobs`, укључујући и превођење посла у статус `completed` када су сви фајлови обрађени.

5.5.2 Структура пројекта

```
suggestion_generator_service/
├─ Cargo.toml
├─ src/
│   ├─ main.rs          – HTTP sloj, orkestracija, upis u MongoDB i
│   └─ PostgreSQL
│       ├─ models.rs    – Language, Suggestion, RankedProblemPayload
│       ├─ rabbitmq.rs   – RabbitMQ konzument (suggest_queue, krajnja tačka
│       │                 pipeline-a)
│       └─ suggestions.rs – SuggestionEngine – šablonski mehanizam
│                           generisanja predloga
```

5.5.3 Шаблонски механизам генерисања предлога

У складу са опредељењем описаним у README документацији пројекта — предлози се генеришу искључиво на основу унапред дефинисаних, детерминистичких шаблона, без употребе LLM модела или анализе ширег контекста кода —

SuggestionEngine::build_suggestion (Листинг 10) препознаје тип проблема поређењем подниске (substring matching) над називом типа проблема, и за сваки препознат тип враћа тројку: текстуално објашњење проблема, предлог исправљеног кода прилагођен језику фајла, и фиксну оцену утицаја (impact_score).

```
fn build_suggestion(&self, p_type: &str, lang: &Language, snippet: &str) ->
(String, String, u8) {
    if p_type.contains("hardcoded_secret") || p_type.contains("hardcoded-
password")
        || p_type.contains("credential") {
        let code = match lang {
            Language::Python => "import os\nsecret =
os.environ[\"SECRET_KEY\"]\n...",
            Language::JavaScript => "import 'dotenv/config';\n...",
            Language::Rust => "dotenvy::dotenv().ok();\nlet secret =
std::env::var(\"SECRET_KEY\")...",
            Language::Java => "@Value(\"${app.secret-key}\")\nprivate
String secretKey; ...",
            Language::Go => "secret := os.Getenv(\"SECRET_KEY\") ...",
            _ => "# Read secrets from environment variables: ...",
        };
        return ("Hardcoded secrets ... are a critical security risk. \
Use environment variables or a secrets manager.".into(),
code.into(), 95);
    }
    // ... još 11 prepoznatih kategorija, pa generički fallback
}
```

Листинг 10: Препознавање типа проблема и генерисање предлога по језику, скраћено (suggestion_generator_service/src/suggestions.rs)

Уколико тип проблема не одговара ниједном препознатом обрасцу (нпр. нов тип који је додао Semgrep, а још није експлицитно покривен), сервис враћа генерички предлог који корисника упућује на опште смернице добре праксе за дати језик, уз средњу оцену утицаја (50), уместо да изостави предлог за тај проблем.

5.5.4 Покривеност типова проблема

Табела 10 приказује свих 12 експлицитно покривених категорија проблема, поређаних по додељеној оцени утицаја (impact_score), која служи као независна, статична процена озбиљности проблема из перспективе предлога исправке (за разлику од динамичке rank_score вредности из поглавља 5.4, која додатно узима у обзир контекст конкретног налаза — позицију у фајлу, изложеност фајла и др.).

Табела 10: Покривене категорије проблема и оцена утицаја предлога

Категорија (препознати обрасци у називу типа)	impact_score
Хардкодоване тајне (hardcoded_secret, hardcoded-password, credential)	95
SQL инјекција (sql_injection, sqlalchemy-execute-raw)	90
Небезбедно генерисање случајних вредности (insecure_random)	88
XSS рањивост (xss)	85
Дуплиран код (duplicate_code, duplication)	80
N+1 / неоптимизован упит (n_plus_one, unoptimized_query)	80
Угњеждена петља (nested_loop, unoptimized_loop)	75
Конкатенација стрингова у петљи (string_concat_in_loop, string_concat)	72
Дубоко угњеждавање / дуга листа параметара (deep_nesting, long_parameter_list)	65
Дуга / комплексна метода (long_method, complex_method)	60
Велика класа / дугачак фајл (large_class, long_file)	58
Магични број (magic_number)	55
(Генерички fallback, нејознајт џиџи)	50

Свих пет детекторских категорија из Analysis сервиса (поглавље 5.3), као и тип проблема који генерише искључиво Semgrep (sql_injection), имају одговарајући шаблон — у тренутној имплементацији не постоји тип проблема који систем детектује, а за који не би понудио бар генерички предлог.

5.5.5 Језичка прилагођеност предлога

За девет од дванаест категорија, предлог кода је засебан за сваки од пет језика са сопственим скупом идиоматских решења (нпр. secrets модул у Pythonу и crypto.getRandomValues() у JavaScript-у за небезбедно генерисање случајних вредности, или strings.Builder у Go-у и StringBuilder у Јави за конкатенацију стрингова у петљи), док се за језике ван овог скупа користи генерички, језички-неутралан предлог.

Поље language у поруци коју Suggestion Generator прима од ML Ranker сервиса је релативно скорашњи додатак pipeline-у — коментар у изворном коду (models.rs) експлицитно наводи да ранија верзија поруке није садржала језик фајла, услед чега су сви предлози, независно од стварног језика проблема, приказивали исти генерички код. Ово је поправљено прослеђивањем језика кроз цео pipeline (Parser → Analysis → ML Ranker → Suggestion Generator), а поље је у DTO структури

(RankedProblemPayload) маркирано подразумеваном вредношћу (Language::Unknown) ради компатибилности уназад, уколико би порука из неког разлога стигла без овог поља.

5.5.6 Модел података

Табела 11: Структура Suggestion (уписује се у MongoDB колекцију suggestions)

Поље	Опис
id	Јединствени идентификатор предлога
problem_id	Референца на проблем из MongoDB колекције problems
analysis_job_id	Референца на посао анализе
problem_type	Тип проблема (наслеђен из Analysis/Semgrep налаза)
explanation	Текстуално објашњење проблема и предложеног решења (на енглеском језику)
original_code	Оригинални исечак кода који садржи проблем
suggested_code	Предложени (исправљени) код, прилагођен језику фајла
impact_score	Статична оцена утицаја предлога (табела 10)
created_at	Временска ознака креирања предлога

5.5.7 Упис резултата и ажурирање статуса посла

Након генерисања предлога за све проблеме из поруке, сервис их уписује у MongoDB (insert_many) и потом, као последњи корак pipeline-а, ажурира запис посла у PostgreSQL табели analysis_jobs (Листинг 11): броји обрађен фајл (processed_files = processed_files + 1) и, уколико је тиме достигнут укупан број фајлова (total_files), преводи статус посла у completed и бележи време завршетка.

```

UPDATE analysis_jobs
SET processed_files = processed_files + 1,
    status = CASE
        WHEN processed_files + 1 >= total_files THEN 'completed'
        ELSE 'processing'
    END,
    completed_at = CASE
        WHEN processed_files + 1 >= total_files THEN NOW()
        ELSE completed_at
    END
WHERE id = $1

```

Листинг 11: Ажурирање напретка и статуса посла, извршава се по сваком обрађеном фајлу (suggestion_generator_service/src/main.rs)

Овакво решење — инкрементално ажурирање једним атомичним SQL упитом по фајлу, уместо накнадног пребројавања свих фајлова посла — избегава потребу да сервис памти или поново учитава укупно стање посла, а CASE израз обезбеђује да се прелазак у статус `completed` деси тачно у тренутку када је и последњи фајл обрађен, без обзира на редослед којим појединачни фајлови (услед конкурентне обраде, поглавље 5.4.7) заврше кроз pipeline.

5.5.8 RabbitMQ интеграција

Suggestion Generator прати идентичан образац као претходна три сервиса: петља са експоненцијалним одлагањем за поновно повезивање и семафор са 20 дозвола за ограничавање конкурентности, са преузимањем порука из `suggest_queue`. Како је ово последњи сервис у pipeline-у, овде се ланац RabbitMQ редова завршава — сервис не објављује даље поруке, већ директно уписује крајњи резултат у базе података које потом чита API Gateway приликом агрегације одговора (поглавље 4.5).

5.6 API Gateway

5.6.1 Одговорност сервиса

API Gateway је једина компонента система која излаже јавни REST интерфејс корисничком интерфејсу. Задужен је за аутентикацију захтева, прихват кода за анализу у три различита облика, покретање pipeline-а слањем порука у `parse_queue`, агрегацију готових резултата из MongoDB и PostgreSQL база, кеширање одговора у Redis, листање историје анализа по кориснику и брисање података на захтев корисника. За разлику од осталих пет сервиса, API Gateway не обавља сопствену пословну логику анализе кода — у том смислу је танак слој (thin layer) који оркестрира остатак система и нормализује приступ ка њему.

5.6.2 Структура пројекта

```
api_gateway/
├── Cargo.toml
└── src/
    ├── main.rs          – иницијализација, регистрација ruta
    ├── routes.rs         – обрађивачи свих HTTP ruta
    ├── middleware.rs     – JWT auth middleware
    ├── scanner.rs        – обилазак директоријума и слање фајлова у
parse_queue
└── rabbitmq.rs          – иницијализација RabbitMQ konekcije, објављивање
poruka
```

5.6.3 REST API

Табела 12 приказује комплетан скуп рута које API Gateway излаже. Руте су подељене на јавне (регистрација и пријава, које се само прослеђују ка Auth Service-u) и заштићене, које захтевају важећи JWT токен.

Табела 12: Руте API Gateway-a

Метод	Путања	Опис	Заштићено
GET	/health	Провера доступности сервиса	Не
POST	/api/auth/register	Прослеђивање ка Auth Service-u	Не
POST	/api/auth/login	Прослеђивање ка Auth Service-u	Не
POST	/api/analyze	Анализа појединачног исечка ко-да (JSON тело)	Да
POST	/api/analyze/git	Анализа јавног GitHub (Git) репо-зиторијума	Да
POST	/api/analyze/zip	Анализа ZIP архиве (multipart upload)	Да
GET	/api/results/:job_id	Статус посла или готови резулта-ти анализе	Да
DELETE	/api/results/:job_id	Трајно брисање свих података по-сла	Да
GET	/api/jobs	Историја анализа улогованог ко-рисника	Да

Руте за регистрацију и пријаву су намерно имплементиране као танак прокси (HTTP позив ка AUTH_SERVICE_URL, уз прослеђивање статуса и тела одговора), уместо да API Gateway сам комуницира са PostgreSQL базом корисника — на тај начин Auth Service остаје једини власник логике аутентикације, у складу са принципом да сваки микро-сервис има једну јасно дефинисану одговорност (поглавље 4.1).

5.6.4 Аутентикација и провера власништва

Заштићене руте пролазе кроз auth_middleware, који чита Authorization: Bearer <token> заглавље, верификује JWT потпис истим тајним кључем који користи Auth Service (JWT_SECRET, поглавље 5.1.7), и при успеху уписује декодоване клејмове (Claims) у екстензије Ахум захтева, одакле их обрађивачи рута даље читају.

Поред провере идентитета, руте које приступају конкретном послу анализе (GET/DELETE /api/results/:job_id) додатно проверавају **власништво** над тим послом — идентификатор корисника из JWT токена се пореди са user_id колоном записа у analysis_jobs табели, и захтев се одбија са статусом 403 уколико се корисник не

поклапа са власником посла. Ова провера је независна од саме аутентикације: важећи токен доказује ко је корисник, док провера власништва одређује да ли тај корисник сме да приступи баш овом ресурсу — разликовање битно за систем у коме више корисника дели исту инстанцу платформе.

5.6.5 Начини уноса кода за анализу

Систем подржава три начина покретања анализе, сва три креирају запис у `analysis_jobs` и покрећу исти `pipeline` (поглавље 4.5), али се разликују по томе како се до скупа фајлова долази:

- **Појединачан исечак кода** (`POST /api/analyze`) — тело захтева већ садржи код и назив језика; сервис креира посао са `total_files = 1` и директно објављује једну поруку у `parse_queue`, без проласка кроз `scanner`.
- **Git репозиторијум** (`POST /api/analyze/git`) — сервис клонира јавни репозиторијум алатом `git2` у привремени директоријум. Како је класично (блокирајуће) I/O клонирања репозиторијума неприкладно за позивање директно унутар асинхроне `Tokio` петље, позив је обавијен у `tokio::task::spawn_blocking`, који га извршава на засебном `thread pool`-у намењеном блокирајућим операцијама, не блокирајући притом остатак асинхроног извршног окружења сервиса.
- **ZIP архива** (`POST /api/analyze/zip`) — сервис прима фајл кроз `multipart/form-data`, распакује га у привремени директоријум ручним обиласком уноса архиве (`zip crate`).

За последња два начина, директоријум се даље прослеђује функцији `scan_directory_and_publish` (`scanner.rs`), која рекурзивно обилази све фајлове, прескаче директоријуме који по конвенцији не садрже кориснички код (`.git`, `node_modules`, `target`, `venv`), одређује језик фајла на основу екстензије, и за сваки препознат фајл објављује засебну поруку у `parse_queue` — ово је тачка у систему на којој настаје шема “једна порука по фајлу” описана у поглављу 4.4.

Фајлови са неподржаном екстензијом, као и фајлови који се не могу прочитати као валидан UTF-8 текст (нпр. бинарни фајлови случајно укључени у репозиторијум), се тихо прескачу — не пријављују се као грешка, већ једноставно не улазе у број послатих фајлова (`total_files`). За разлику од уноса појединачног исечка кода, где корисник експлицитно наводи језик, код `git`/`ZIP` уноса се у `analysis_jobs` табели колона `language` уписује као фиксна вредност `"mixed"`, с обзиром да репозиторијум по правилу садржи фајлове на више језика.

5.6.6 Агрегација резултата и кеширање у Redis

`GET /api/results/:job_id` прво проверава статус посла у PostgreSQL бази — уколико посао још није завршен (`processed_files < total_files`), одговор кориснику јавља тренутни напредак (`"Analyzing codebase... (7/23) files processed"`) без

приступања MongoDB бази. Тек када је посао завршен, сервис прелази на агрегацију стварних резултата (Листинг 12): прво проверава Redis кеш под кључем `results:: {job_id}`, и при поготку (cache hit) одмах враћа кеширан JSON одговор без иједног упита ка MongoDB бази.

```
let cache_key = format!("results::{}", job_id);
if let Ok(mut con) = state.redis_client.get_async_connection().await {
    let cached: redis::RedisResult<Option<String>> = redis::cmd("GET")
        .arg(&cache_key).query_async(&mut con).await;
    if let Ok(Some(json_str)) = cached {
        if let Ok(val) = serde_json::from_str::<Value>(&json_str) {
            return Ok((StatusCode::OK, Json(val))); // cache HIT
        }
    }
}
// ... cache MISS: agregacija iz MongoDB (problems + suggestions
// kolekcije) ...
if let Ok(mut con) = state.redis_client.get_async_connection().await {
    let _: redis::RedisResult<()> = redis::cmd("SETEX")
        .arg(&cache_key).arg(3600u32).arg(&json_str).query_async(&mut
con).await;
}
```

Листинг 12: Redis кеширање комплетних резултата посла, скраћено (`api_gateway/src/routes.rs`)

Уколико Redis привремено није доступан, грешка се не прослеђује кориснику као неуспех захтева (best-effort приступ) — сервис једноставно наставља директно ка MongoDB бази, као да је дошло до промашаја кеша (cache miss), а упис у кеш при крају обраде се на исти начин тихо прескаче ако Redis није доступан. Резултат се чува уз рок истека од 3600 секунди (SETEX), након чега се, при следећем захтеву, поново генерише из MongoDB базе — довољно дуго да опслужи поновљене прегледе истог резултата у току једне сесије рада корисника, а довољно кратко да не захтева експлицитну инвалидацију кеша осим при брисању посла (поглавље 5.6.8).

При агрегацији, сервис из колекције `problems` чита и `rank_score` који је уписао ML Ranker; уколико тај податак из неког разлога недостаје (нпр. теоријска могућност да је посао означен завршеним пре него што је рангирање стигло), користи се резервна (fallback) вредност изведена искључиво из основне Severity оцене, чиме се одговор кориснику не руши у одсуству ранга, већ деградира на једноставнији, али функцио-налан приказ.

5.6.7 Историја анализа

GET `/api/jobs` чита до 50 последњих записа из `analysis_jobs` табеле за тренутно улогованог корисника (`WHERE user_id = $1 ORDER BY created_at DESC`), и

представља имплементацију функционалности “историја анализа” најављене у поглављу 4.6 — свака ставка носи статус, језик, назив фајла/репозиторијума, напредак (`processed_files/total_files`) и временске ознаке, довољно за приказ листе претходних анализа корисника у корисничком интерфејсу без потребе да се за сваку ставку посебно упитује MongoDB.

5.6.8 Брисање података корисника

`DELETE /api/results/:job_id` имплементира трајно брисање свих података везаних за један посао анализе, након провере власништва идентичне оној у поглављу 5.6.4. Брисање се спроводи у три корака: (1) брисање реда из `analysis_jobs` табеле у PostgreSQL, где `ON DELETE CASCADE` страна веза (поглавље 4.6) аутоматски уклања све зависне редове; (2) ручно брисање одговарајућих докумената из MongoDB колекција `problems`, `suggestions` и `parsed_ast` по `analysis_job_id`, с обзиром да MongoDB нема механизам страних веза који би ово урадио аутоматски; (3) инвалидација (брисање) Redis кеш записа за тај посао, како наредни покушај читања обрисаног посла не би случајно вратио застарео кеширан одговор.

Ова рута у изворном коду експлицитно носи напомену да представља подршку за право корисника на брисање података. Важно је прецизно разграничити домет ове функционалности у односу на формулацију из поглавља 3.5 и README документације пројекта: систем не тврди пуну, формалну GDPR усклађеност (која би обухватала и теме попут евиденције обраде података, рокова чувања, извоза података и др.), али имплементира конкретан механизам права на брисање (`right to erasure`), доступан сваком кориснику над сопственим подацима, са провером власништва и доследним чишћењем свих база у којима се подаци тог посла налазе.

5.6.9 Поузданост повезивања — напомена о асиметрији

За разлику од четири `worker` сервиса (`Parser`, `Analysis`, `ML Ranker`, `Suggestion Generator`), који се повезују на RabbitMQ кроз петљу са експоненцијалним одлагањем и настављају покушаје поновног повезивања у недоглед (поглавље 5.2.6), API Gateway се повезује на RabbitMQ само једном, приликом покретања сервиса (`setup_rabbitmq`), и у случају неуспеха одмах прекида рад сервиса. Ово је намерна асиметрија, а не превид: Docker Compose конфигурација сервиса (поглавље 3.5.3) експлицитно наводи `depends_on` услов `service_healthy` над RabbitMQ, PostgreSQL и MongoDB, чиме се, у контролисаном окружењу за демонстрацију, гарантује да су ове зависности већ здраве пре него што се API Gateway уопште покрене — потреба за интерном логиком поновног повезивања при старту је тиме пребачена на ниво оркестрације контејнера. Уколико би веза са RabbitMQ-ом била прекинута након успешног старта сервиса, исти ризик остаје и за API Gateway као и за остале сервисе, с обзиром да тренутна имплементација не покушава поновно повезивање над већ успостављеним, а потом изгубљеним, каналом.

5.6.10 Ограничења

У обради мултипарт ZIP уpload захтева (`analyze_zip_handler`) присутан је мањи број позива `.unwgar()` над операцијама које зависе од садржаја који доставља корисник (читање поља мултипарт форме, распакивање појединачних уноса архиве, упис распакованих фајлова на диск). Уколико корисник пошаље неисправан или оштећен ZIP фајл, овакав позив доводи до панике (`panic`) унутар асинхроног задатка који опслужује тај конкретан захтев, уместо да врати структурисан HTTP одговор са статусом 400. Замена ових позива одговарајућом обрадом грешака (`Result` пропагација са јасним порукама кориснику) наведена је као ставка за дораду у оквиру даљег развоја (поглавље 8).

5.7 Веб кориснички интерфејс

5.7.1 Одговорност и технологије

Веб кориснички интерфејс је једнострана React апликација која кориснику омогућава регистрацију/пријаву, покретање анализе на три начина која подржава API Gateway (поглавље 5.6.5), праћење напретка посла у току, преглед и филтрирање пронађених проблема, увид у предложене исправке и приступ историји ранијих анализа. Технолошки избори су детаљније описани у поглављу 3.4; овде се описује како су те технологије примењене у конкретним деловима интерфејса.

5.7.2 Структура пројекта

```
frontend/src/
├── App.tsx                – definicija ruta (react-router)
├── context/AuthContext.tsx – globalno stanje autentikacije
├── components/ProtectedRoute.tsx – čuvar zaštićenih ruta
├── services/api.ts        – Axios instance, interceptori
├── pages/
│   ├── Login.tsx
│   ├── Register.tsx
│   ├── Dashboard.tsx      – pokretanje analize, istorija
│   └── Result.tsx         – prikaz rezultata analize
```

5.7.3 Аутентикација на клијентској страни

Стање аутентикације (JWT токен, подаци о кориснику) чува се у React контексту (`AuthContext`) и перзистира у `localStorage` читача, тако да корисник остаје пријављен и након освежавања странице. `ProtectedRoute` компонента, постављена као родитељ свих заштићених рута у `App.tsx`, преусмерава неаутентикованог корисника на `/login`, док током иницијалне провере стања приказује индикатор учитавања уместо да кратко “бљесне” пријавна страница пре него што се стање аутентикације учита из `localStorage`.

Axios инстанца у `services/api.ts` користи два `interceptor`-а: захтевни (`request`) `interceptor` аутоматски додаје `Authorization: Bearer <token>` заглавље на сваки одлазни захтев уколико токен постоји, док одговорни (`response`) `interceptor` препознаје HTTP статус 401 (токен истекао или неважећи) и аутоматски одјављује корисника, бришући токен из `localStorage` и преусмеравајући га на пријаву — чиме се свака компонента која позива API ослобађа потребе да сама рукује овим случајем.

5.7.4 Dashboard — покретање анализе

Dashboard страница нуди три картице које одговарају трима начинима уноса описаним у поглављу 5.6.5: налепљивање исечка кода уз избор језика (позив `POST /api/analyze`), унос URL-а јавног GitHub репозиторијума (`POST /api/analyze/git`), и превлачење (`drag-and-drop`) или избор ZIP архиве (`POST /api/analyze/zip`, као `multipart/form-data`). Након успешног покретања посла, корисник се аутоматски преусмерава на страницу резултата за тај посао.

Ради лакшег упознавања са алатом, картица за унос исечка кода садржи унапред припремљене примере кода по језику, са намерно уграђеним проблемима које систем препознаје (нпр. хардкодован API кључ, упит осетљив на SQL инјекцију и конкатенација стрингова у петљи у Python примеру; небезбедно генерисање случајних вредности у JavaScript примеру) — корисник тако може одмах, једним кликом, видети алат у акцији без потребе да сам смишља проблематичан код.

Бочна трака Dashboard-а приказује историју анализа корисника (`GET /api/jobs`, поглавље 5.6.7), са статусом, језиком и временом сваке раније анализе, као и дугметом за трајно брисање појединачног посла (`DELETE /api/results/:job_id`, поглавље 5.6.8).

5.7.5 Приказ резултата и праћење напретка

Како се анализа извршава асинхроно кроз микросервисни `pipeline` (поглавље 4.5), Result страница не приказује резултат одмах, већ периодично (на сваке 3 секунде) поново позива `GET /api/results/:job_id` све док статус посла не постане `COMPLETED` или `FAILED`, приказујући у међувремену текст напретка који враћа API Gateway (нпр. број обрађених од укупног броја фајлова). Овакав приступ праћења стања периодичним поновним упитом (`polling`) је једноставнији за имплементацију од алтернатива заснованих на трајној вези (`WebSocket`, `Server-Sent Events`), по цену мање ефикасности при врло дугим анализама или великом броју истовремених корисника — разматран је као могући правац унапређења у поглављу 8.

Када је посао завршен, страница приказује: кружни (`donut`) график расподеле проблема по озбиљности (библиотека `recharts`), листу проблема груписану по фајлу и сортирану по `rank_score` (поглавље 5.4), са могућношћу филтрирања по нивоу озбиљности; и, за изабран проблем, Monaco Editor компоненту за приказ оригиналног кода око места проблема. Уколико за проблем постоји предлог исправке, користи се Monaco-

ova DiffEditor компонента, која оригинални и предложени код (`original_code/suggested_code`, поглавље 5.5.6) приказује упоредо, са визуелно истакнутим разликама — исти приказ који програмери познају из code review алата попут GitHub-а или из самог VS Code едитора.

5.7.6 Ограничења

JWT токен се чува у `localStorage`, што је једноставно за имплементацију и довољно за потребе демонстрације, али је, за разлику од приступа који користи `httpOnly` колачиће, у принципу изложенији крађи токена путем XSS напада, уколико би се таква рањивост појавила негде у апликацији. Замена овог механизма је наведена као могући правац унапређења у поглављу 8, заједно са заменом `polling` приступа из поглавља 5.7.5 механизмом заснованим на трајној вези.

5.8 CLI алат

5.8.1 Одговорност алата

Поред Веб корисничког интерфејса, платформа нуди и команднолинијски (CLI) алат `hero-optimizer`, намењен сценаријима у којима покретање пуне Веб апликације није практично или пожељно — локална анализа кода директно из развојног окружења, као и уклапање анализе у аутоматизоване CI/CD цевоводе (нпр. као корак у `pipeline`-у који анализира измењени код при сваком `pull request`-у). CLI алат не представља алтернативни пут обраде — намерно је пројектован да користи **исте REST руте API Gateway-а** као и Веб интерфејс (поглавље 5.6.3), тако да је понашање система, укључујући правила детекције, рангирање и предлоге исправки, идентично независно од тога да ли је анализа покренута из прегледача или са командне линије.

5.8.2 Структура пројекта

CLI је, као и шест микросервиса, део истог `Cargo workspace`-а (поглавље 3.5.1), као засебан бинарни пакет ван `services/` директоријума:

```
cli/
├─ Cargo.toml
├─ src/
│   ├── main.rs          – parsiranje argumenata i komandi (clap)
│   ├── config.rs        – čuvanje/učitavanje JWT tokena (~/.repo-optimizer/
config.toml)
│   ├── scanner.rs       – obilazak lokalnog direktorijuma i pakovanje u ZIP
│   ├── client.rs        – HTTP klijent ka API Gateway-u
│   └─ report.rs         – formatiranje izlaza (terminal / JSON)
```

5.8.3 Команде

Табела 13 приказује доступне команде алата, сваку повезану са одговарајућом рутом API Gateway-а.

Табела 13: Команде CLI алата

Команда	Рута API Gateway-а	Опис
repo-optimizer login	POST /api/auth/login	Интерактивна пријава (мејл/лозинка), локално чување JWT токена
repo-optimizer analyze <putanja>	POST /api/analyze/zip	Анализа локалног директоријума или фајла
repo-optimizer results <job_id>	GET /api/results/:job_id	Приказ резултата раније покренуте анализе
repo-optimizer history	GET /api/jobs	Преглед историје анализа корисника

5.8.4 Аутентикација

Како све руте за анализу на API Gateway-у захтевају важећи JWT токен (поглавље 5.6.4), CLI при првој употреби захтева извршавање команде login, која корисничке податке прослеђује ка POST /api/auth/login на идентичан начин као пријавна форма Веб интерфејса (поглавље 5.7.3), а добијени токен чува локално, у конфигурационом фајлу у корисничком home директоријуму. Наредне команде аутоматски читају сачуван токен и прилажу га као Authorization: Bearer заглавље, без потребе да га корисник поново уноси при сваком позиву — исти принцип као Axios interceptor на Веб страни (поглавље 5.7.3), само примењен кроз локални конфигурациони фајл уместо localStorage-а прегледача.

5.8.5 Ток извршавања анализе

Команда analyze следи тачно исти ток као и унос ZIP архиве кроз Веб интерфејс (поглавље 5.7.4), са том разликом да се паковање у ZIP архиву дешава локално, на машини корисника, уместо да корисник то ради ручно пре отпремања:

1. scanner.rs рекурзивно обилази задату путању, изостављајући исте директоријуме које изоставља и scan_directory_and_publish на API Gateway-у (.git, node_modules, target, venv — поглавље 5.6.5), ради доследног понашања независно од тога да ли се скенирање дешава локално (CLI) или на серверу (Веб ZIP унос);
2. резултат се пакује у ZIP архиву у меморији и шаље на POST /api/analyze/zip, на исти начин као и при отпремању из прегледача;
3. CLI затим периодично (на сваке 3 секунде — идентичан интервал као Result страница Веб интерфејса, поглавље 5.7.5) позива GET /api/results/:job_id и у терминалу освежава текст напретка који враћа API Gateway, све док статус не постане COMPLETED или FAILED;

4. по завршетку, резултати се исписују као форматиран извештај у терминалу — проблеми груписани по фајлу, сортирани по `rank_score` (поглавље 5.4), са нивоом озбиљности означеним бојом текста (ANSI escape секвенце: црвена за `Critical`, жута за `High`, и др.) — терминалски еквивалент бојених ознака озбиљности из Веб интерфејса.

5.8.6 Излазни формат и интеграција у CI/CD

Поред подразумеваног, читљивог терминалског извештаја, команде `analyze` и `results` подржавају опцију `--json`, која уместо форматираног текста исписује сиров JSON одговор API Gateway-а на стандардни излаз. Овај облик излаза је намењен аутоматизованој обради — нпр. кораку у CI/CD цевоводу који резултат анализе прослеђује другом алату или на основу броја пронађених проблема високе озбиљности одлучује да ли да означи `build` као неуспешан — чиме CLI алат испуњава улогу најављену у правцима даљег развоја README документације пројекта, као начин интеграције платформе без ослањања на Веб интерфејс.

6.1 Приступ тестирању

Квалитет backend кода проверен је искључиво јединичним (unit) тестовима, писаним уз употребу Rust-ovog уграђеног тест framework-а (`#[test]` атрибут, покретање командом `cargo test`), без додатног екстерног оквира за тестирање. Тестови су смештени непосредно уз код који тестирају, у модулу означеном атрибутом `#[cfg(test)]` на дну сваког изворног фајла — конвенција коју Rust заједница преферира, с обзиром да тестови тако имају приступ и приватним (не само јавним) функцијама и структурама модула, а `#[cfg(test)]` обезбеђује да се тест код уопште не компајлира у финални (release) бинарни фајл.

Укупан број јединичних тестова у пројекту износи 62, распоређених по сервисима као у табели 14.

Табела 14: Преглед јединичних тестова по сервису

Сервис	Број тестова	Фокус тестирања
Analysis Service	43	Детектори (code smell, перформансе, безбедност, дупликација), дедупликација налаза
Parser Service	12	Извлачење функција/класа из AST-а (Go, Java, Rust)
ML Ranker Service	7	Формула рангирања, калибрација озбиљности, хијерархија категорија
Auth Service	0	—
Suggestion Generator Service	0	—
API Gateway	0	—

Тестирање је, у тренутној фази пројекта, намерно усмерено на сервисе са највише самосталне, детерминистички проверљиве пословне логике (детекција проблема, парсирање, рангирање) — сервиси који су претежно танки слојеви над HTTP-ом, базом података или спољним сервисима (Auth, Suggestion Generator, API Gateway) тренутно немају сопствене јединичне тестове, што је наведено као ограничење у поглављу 6.6, а не као превид који би требало да остане неописан.

6.2 Шаблон тестних података (test fixtures)

Да би се избегло понављање исте дугачке иницијализације у сваком тесту, тестови у Analysis сервису доследно користе једноставан образац “подразумевано па изузетак” (Листинг 13): помоћна функција враћа валидну, “чисту” инстанцу улазних података (функцију без иједног проблема), а сваки тест мења само оно поље које је релевантно за случај који проверава, пре него што га проследи детектору.

```
fn base_function() -> FunctionInfo {
    FunctionInfo {
        name: "test_fn".into(),
        lines_of_code: 10,
        parameter_count: 2,
        nesting_depth: 1,
        cyclomatic_complexity: 3,
        // ... ostala polja sa "bezbednim" podrazumevanim vrednostima
    }
}

fn make_function(f: FunctionInfo) -> ParsedAst { /* umotava FunctionInfo u
ParsedAst */ }
```

Листинг 13: Образац тестних података “подразумевано па изузетак” (analysis_service/src/detectors/smells.rs)

Захваљујући овом обрасцу, тест попут оног у поглављу 6.3 своди се на једну измењену вредност поља (f.lines_of_code = 51), чиме је намера теста читљива на први поглед, без потребе да читалац размрси које су вредности случајне, а које битне за проверу.

6.3 Гранични (boundary) тестови

За правила заснована на нумеричким праговима (табела 7), тестови доследно проверавају понашање тачно на граници и непосредно преко ње, а не само “очигледне” случајеве дубоко унутар или ван опсега. Пример за правило “дуга метода” (праг 50 линија):

```

#[test]
fn long_method_fires_above_50_loc() {
    let mut f = base_function();
    f.lines_of_code = 51;
    let problems = SmellDetector::new().detect(&make_function(f));
    assert!(problems.iter().any(|p| p.problem_type ==
"code_smell.long_method"));
}

#[test]
fn long_method_does_not_fire_at_50_loc() {
    let mut f = base_function();
    f.lines_of_code = 50;
    let problems = SmellDetector::new().detect(&make_function(f));
    assert!(!problems.iter().any(|p| p.problem_type ==
"code_smell.long_method"));
}

```

Листинг 14: Гранични тест услова “> 50 линија” (analysis_service/src/detectors/smells.rs)

Пар тестова попут овог експлицитно потврђује да је услов у коду заиста строго веће (> 50), а не веће-или-једнако — разлика која се лако погрешно напише, а чију би последицу (правило које се окида један ред раније или касније него што је намеравано) обичан “срећни пут” тест не би открио. Исти образац се доследно понавља за све остале прагове у табели 7 (параметри, угњеждавање, комплексност, величина класе/фајла), као и за праг озбиљности (нпр. `long_method_severity_high_above_100_loc`, који проверава да озбиљност прескочи на High тачно преко другог прага).

6.4 Регресиони тестови

Више тестова у Analysis сервису је експлицитно везано за конкретан, раније уочен и исправљен пропуст у детекцији (поглавље 5.3.5, 5.3.6), а не за опште понашање правила. Тест `detects_aws_secret_key_regression` (Листинг 15) је најдиректнији пример:

```
#[test]
fn detects_aws_secret_key_regression() {
    let code = "aws_secret_key = \"wJalrXUtnFEMI/K7MDENG/
bPxRfiCYEXAMPLEKEY\"\\n";
    let problems = SecurityDetector::new().detect(&ast_with_code(code,
Language::Python));
    assert!(
        problems.iter().any(|p| p.problem_type ==
"security.hardcoded_secret"),
        "aws_secret_key mora biti detektovan kao hardkodovan secret"
    );
}
```

Листинг 15: Регресиони тест за исправљен образац препознавања AWS креденцијала (analysis_service/src/detectors/security.rs)

Овакви тестови имају другачију сврху од граничних тестова из поглавља 6.3 — не проверавају границу правила, већ конкретан улаз који је у ранијој верзији регеха пролазио недетектован, и тиме трајно штите од поновног увођења исте грешке при будућим изменама обрасца. Исти приступ је примењен и за детекцију N+1 упита (no_false_positive_when_select_is_outside_any_loop, поглавље 5.3.5), где тест проверава да SQL упит дефинисан изван било које петље више не генерише лажан налаз, што је био недостатак раније, поједностављене имплементације.

6.5 Тестирање модела рангирања

Како ML Ranker Service (поглавље 5.4) није тренирани модел, уобичајене мере попут прецизности (ассурасу) или F1 мере нису примењиве — уместо тога, тестови проверавају да формула рангирања задовољава скуп пословно смислених инваријанти, односно да се рангирање понаша у складу са намером дизајна, без обзира на тачне нумеричке вредности коефицијената. Листинг 16 приказује тест који проверава да хијерархија тежина категорија (табела 9) остаје строго опадајућа.


```
#[test]
fn category_hierarchy() {
    let engine = MLEngine::new();
    assert!(engine.category_weight("security.hardcoded_secret") >
engine.category_weight("security.sql_injection"));
    assert!(engine.category_weight("security.sql_injection") >
engine.category_weight("security.insecure_random"));
    assert!(engine.category_weight("security.insecure_random") >
engine.category_weight("performance.n_plus_one"));
    assert!(engine.category_weight("performance.n_plus_one") >
engine.category_weight("code_smell.duplicate_code"));
    assert!(engine.category_weight("code_smell.duplicate_code") >
engine.category_weight("code_smell.magic_number"));
}
```

Листинг 16: Тест монотоности хијерархије категорија (ml_ranker_service/src/ml_engine.rs)

Слично, тест `severity_calibration_bands` проверава да функција калибрације (поглавље 5.4.6) сваки од четири нивоа озбиљности мапира у тачно одређен, међусобно раздвојен опсег (`Critical` ≥ 0.80 ; `High` у $[0.58, 0.80)$; `Medium` у $[0.30, 0.58)$; `Low` у $[0.08, 0.30)$) и да је поредак строго опадајући ($c > h > m > l$), док тестови `hardcoded_secret_scores_highest`, `security_outranks_style` и `main_file_higher_than_test_file` проверавају да крајња формула рангирања (поглавље 5.4.5), укључујући и мултипликативни фактор изложености фајла, даје очекиван релативни поредак на реалистичним примерима проблема — нпр. да исти тип проблема у `main.rs` добија већу оцену него идентичан проблем у `test_main.rs`, а тест `scores_in_unit_range` проверава да крајња оцена, независно од комбинације улазних обележја, никада не изађе из опсега $[0, 1]$.

6.6 Ограничења приступа тестирању

Ради транспарентности, потребно је навести границе тренутног приступа тестирању:

- **Нема јединичних тестова за Auth, Suggestion Generator и API Gateway сервисе.** Ови сервиси су, за разлику од Analysis/Parser/ML Ranker, претежно организовани око I/O операција (HTTP позиви, упити ка бази, RabbitMQ) које захтевају mock или тест-контејнер инфраструктуру да би се тестирале изоловано; та инфраструктура није успостављена у тренутној фази пројекта. Шаблонски механизам Suggestion Generator сервиса (поглавље 5.5.3), иако чисто детерминистички и лако тестибилан без спољних зависности, такође тренутно нема покривеност тестовима.
- **Нема интеграционих тестова.** Сви постојећи тестови проверавају појединачне функције/модуле у изолацији (нпр. један детектор над ручно конструисаним `ParsedAst`), али ниједан тест тренутно не покреће више сервиса заједно нити прове-

- рава стварну размену порука преко RabbitMQ-а, уписивање у праве инстанце база података, или цео ток података описан у поглављу 4.5 од краја до краја (end-to-end).
- **Нема аутоматизованог CI pipeline-а** који би тестове покретао при свакој измени кода — тестови се у тренутној фази покрећу ручно, командом `cargo test`, по појединачном сервису или над целим workspace-ом.

Ови недостаци су свесно остављени ван оквира дипломског рада и наведени су као правци даљег развоја у поглављу 8, уместо да буду приказани као већ решени.

Глава 7

Демонстрација

У овом поглављу је приказан типичан ток коришћења платформе, од покретања система до прегледа предлога исправки, као и кратка демонстрација CLI алата. Снимци екрана су настали извршавањем описаног сценарија над покренутим системом.

7.1 Припрема окружења

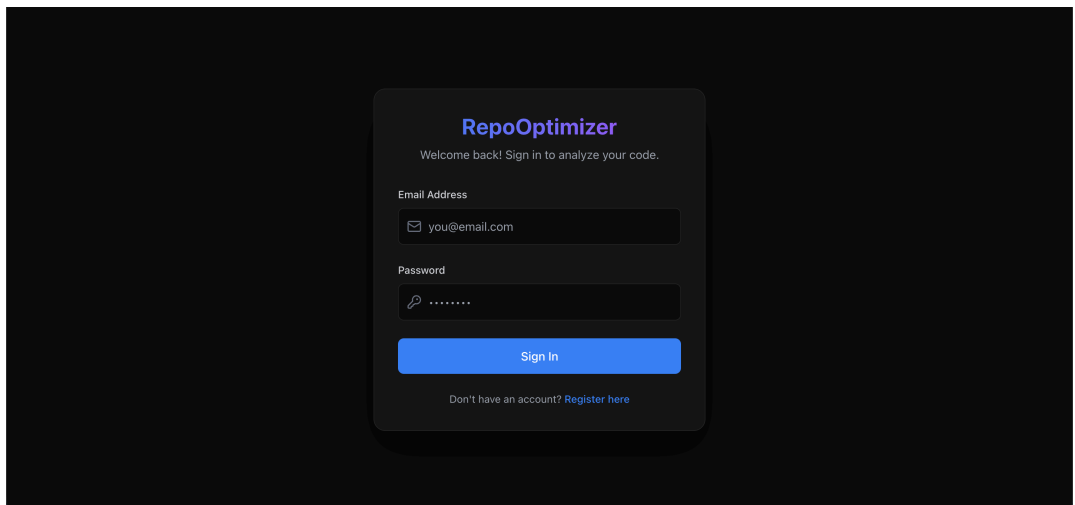
Систем се покреће једном командом из кореног директоријума репозиторијума:

```
docker compose up
```

Након што `docker compose` јави да су сви контејнери здрави (`healthy`) — редослед покретања је гарантован преко `depends_on` услова описаних у поглављу 3.5.3 — Веб интерфејс и API Gateway постају доступни.

7.2 Пријава

На почетној страници корисник уноси креденцијале налога (Слика 2) и бира “Sign In”. Захтев иде на `POST /api/auth/login` (поглавље 5.1.6), а по успешној пријави корисник се преусмерава на Dashboard, док се JWT токен чува у `localStorage` (поглавље 5.7.3).



Слика 2: Пријавна страница Веб интерфејса

7.3 Покретање анализе

На Dashboard-у се бира картица “Code Snippet” и језик Python, након чега се уноси исечак кода (Слика 3):

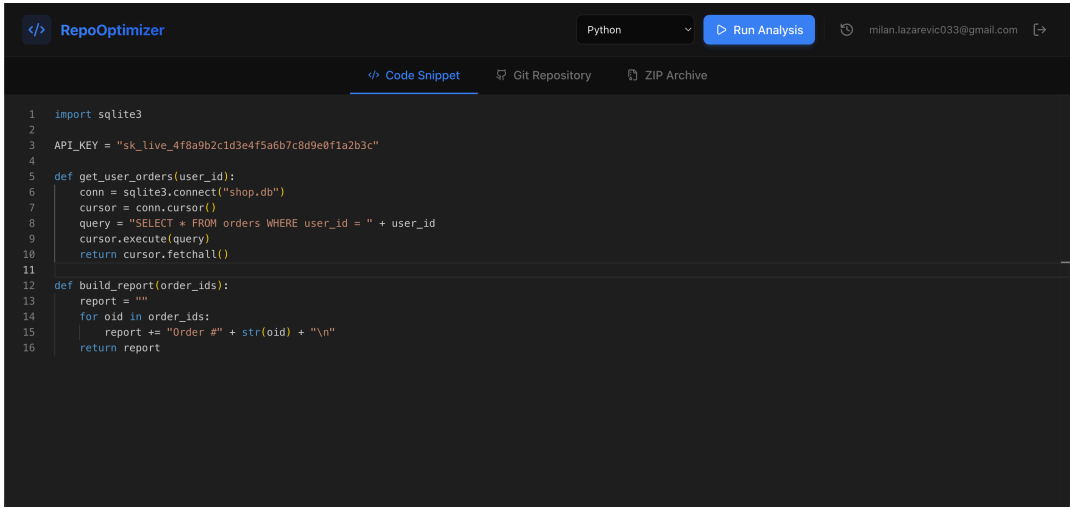
```
import sqlite3

API_KEY = "sk_live_4f8a9b2c1d3e4f5a6b7c8d9e0f1a2b3c"

def get_user_orders(user_id):
    conn = sqlite3.connect("shop.db")
    cursor = conn.cursor()
    query = "SELECT * FROM orders WHERE user_id = " + user_id
    cursor.execute(query)
    return cursor.fetchall()

def build_report(order_ids):
    report = ""
    for oid in order_ids:
        report += "Order #" + str(oid) + "\n"
    return report
```

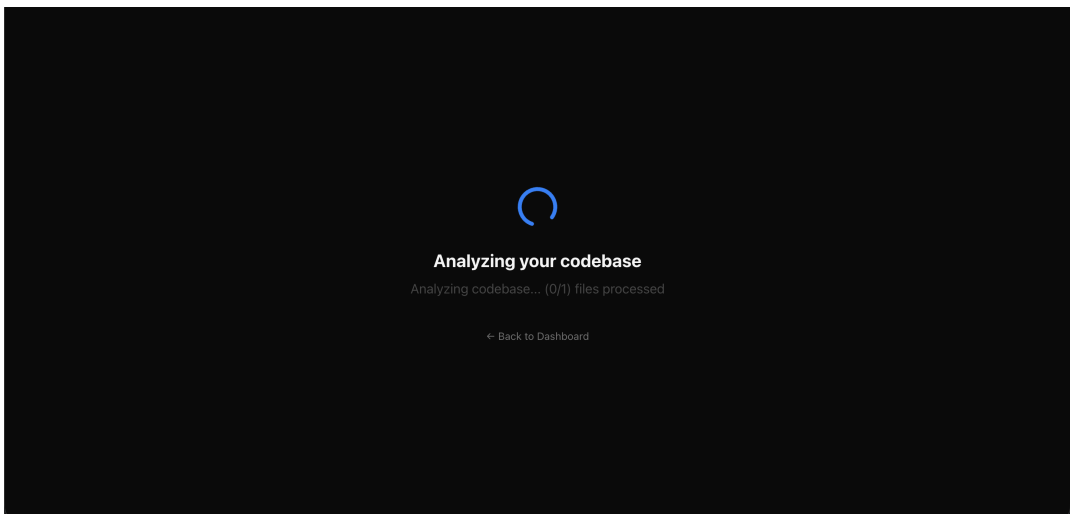
Овај пример намерно садржи три препознатљива проблема описана у поглављу 5.3: хардкодован API кључ (`security.hardcoded_secret`), SQL упит састављен конкатенацијом стрингова без параметризације (`security.sql_injection`, препознат од стране Semgrep-а, поглавље 5.3.6) и конкатенацију стрингова унутар петље (`performance.string_concat_in_loop`, поглавље 5.3.5). Након избора “Run Analysis”, захтев се шаље на `POST /api/analyze` (поглавље 5.6.5), а корисник се преусмерава на страницу резултата за добијени `analysis_job_id`.



Слика 3: Dashboard са унетим примером кода пре покретања анализе

7.4 Праћење напретка

Страница резултата одмах по отварању почиње периодично да позива `GET /api/results/:job_id` (поглавље 5.7.5) и приказује текст напретка који враћа API Gateway (Слика 4). Како пример садржи само један фајл, фаза обраде је кратка.



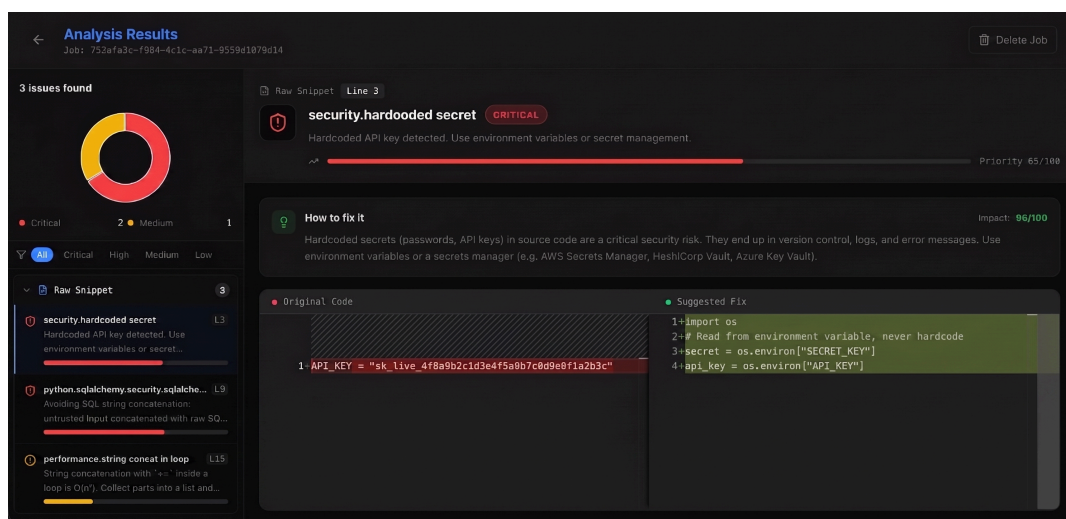
Слика 4: Праћење напретка анализе

7.5 Преглед резултата и предлог исправке

По завршетку посла, страница приказује кружни график расподеле проблема по озбиљности, листу проблема сортирану по `rank_score`, и за изабран проблем — објашњење и предлог исправке кроз Монасо DiffEditor (Слика 5, поглавље 5.7.5).

Над примером из поглавља 7.3, систем је пронашао тачно три проблема: два оцењена као Critical (хардкодован кључ и SQL инјекција, коју је препознао Semgrep под правилом `python.sqlalchemy.security.sqlalchemy-execute-raw-query` — поглавље 5.3.8) и један као Medium (конкатенација стрингова у петљи).

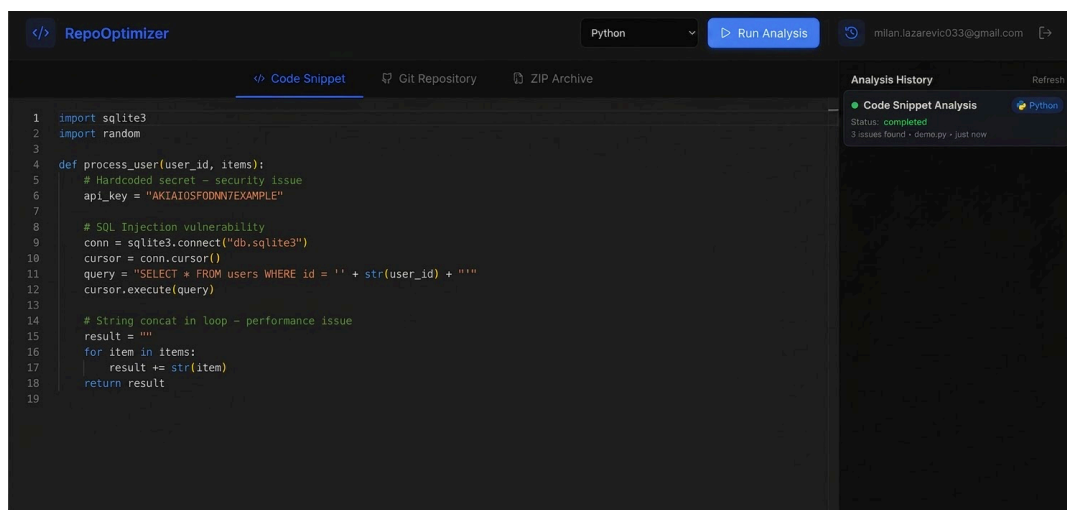
За налаз `security.hardcoded_secret`, систем је доделио `rank_score` 65/100 (“Priority 65/100”) и статичан `impact_score` 95/100 — потоња вредност се тачно поклапа са вредношћу из табеле 10 (поглавље 5.5.4). Предлог исправке (поглавље 5.5.3) упућује на читање тајне из `environment` променљиве уместо директног уписа у изворни код, уз текстуално објашњење ризика (“Hardcoded secrets... are a critical security risk... Use environment variables or a secrets manager”).



Слика 5: Преглед резултата

7.6 Историја анализа

Након извршене анализе, посао се појављује у историји корисника (`GET /api/jobs`, поглавље 5.6.7), са називом, статусом, језиком и временом извршавања (Слика 6).



Слика 6: Бочна трака са историјом анализа корисника

Брисање посла (DELETE /api/results/:job_id, поглавље 5.6.8) се покреће из исте листе; након потврде, ставка нестаје из историје, а поновни покушај приступа истом job_id-у враћа грешку, с обзиром да су сви повезани записи у PostgreSQL, MongoDB и Redis бази обрисани.

7.7 Анализа путем CLI алата

Исти пример кода, сачуван локално као demo.py, може се анализирати и без Веб интерфејса, коришћењем CLI алата описаног у поглављу 5.8. Поруке алата су на енглеском језику, доследно остатку корисничког интерфејса:

```
$ repo-optimizer login
Email: user@example.com
Password: *****
✓ Signed in. Token saved to ~/.repo-optimizer/config.toml

$ repo-optimizer analyze ./demo.py
✓ Packed 1 file, uploading...
[?] Analyzing... (0/1 files processed)
[?] Analyzing... (1/1 files processed)
✓ Analysis complete (job 753afa3c-...)
```

demo.py		
[CRITICAL]	security.hardcoded_secret	line 3
Hardcoded API key detected. Use environment variables...		
[CRITICAL]	security.sql_injection	line 8
Avoiding SQL string concatenation...		
[MEDIUM]	performance.string_concat_in_loop	line 14
String concatenation with `+=` inside a loop is O(n^2)...		

```
$ repo-optimizer analyze ./demo.py --json > result.json
```

Листинг 17: Илустративан ток извршавања CLI алата над примером из поглавља 7.3

Резултат је, с обзиром да CLI алат користи исте руте API Gateway-а као и Веб интерфејс (поглавље 5.8.1), идентичан ономе из поглавља 7.5 — разлика је искључиво у облику приказа (терминалски текст уместо графичког интерфејса), док је последња команда пример излаза погодног за даљу аутоматску обраду (`--json`, поглавље 5.8.6), нпр. у оквиру CI/CD цевовода.

8.1 Преглед урађеног

У оквиру овог рада пројектована је и имплементирана платформа RepoOptimizer — дистрибуиран систем за аутоматизовану статичку анализу изворног кода кроз шест програмских језика (Python, JavaScript, Rust, Java, Go). Систем је реализован као шест независних микросервиса написаних у језику Rust, међусобно повезаних асинхронном разменом порука преко RabbitMQ-а: Auth Service за аутентикацију, Parser Service који коришћењем заједничког Tree-sitter слоја генерише апстрактно синтаксно стабло за сваки подржани језик, Analysis Service који над тим стаблом примењује интерне детекторе из четири категорије (code smell, перформансе, безбедност, дуплиран код) у комбинацији са алатом Semgrep, ML Ranker Service који проблемима додељује оцену озбиљности на основу детерминистички калибрисане формуле, и Suggestion Generator Service који за сваки проблем генерише конкретан, језички прилагођен предлог исправке. Над овим сервисима изграђен је API Gateway који обједињује приступ систему, уз Веб кориснички интерфејс (React) и CLI алат који над истим REST рутама нуде анализу без потребе за графичким интерфејсом. Систем је контејнеризован и покреће се једном Docker Compose командом, а кључна пословна логика покривена је са 62 јединична теста.

Поред основног тока анализе, имплементиране су и пратеће функционалности неопходне за систем са више корисника: праћење статуса посла и историја ранијих анализа по кориснику (одвојена релациона евиденција у PostgreSQL бази, независна од неструктурираних резултата у MongoDB), као и механизам за трајно брисање података корисника на захтев.

8.2 Предности решења

У односу на постојећа решења описана у поглављу 2, RepoOptimizer се истиче комбинацијом својстава која се у пракси ретко срећу заједно у једном алату:

- **Јединствен слој парсирања.** Подршка за шест језика је остварена кроз заједнички Tree-sitter слој и LanguageParser/Detector апстракције (поглавље 5.2.3, 5.3.3), уместо дуплирања логике по језику, чиме је додавање новог језика сведено на јасно омеђен задатак.

- **Конкретни предлози, не само налази.** За разлику од алата који корисника остављају са листом упозорења, сваки пронађен проблем прати и предлог исправке прилагођен језику и конкретном типу проблема (поглавље 5.5).
- **Објашњиво рангирање.** Оцена озбиљности није “црна кутија” — заснива се на јасно дефинисаним, документованим тежинама по категорији и контексту (поглавље 5.4), а сваки рангиран проблем носи и вектор доприноса обележја, што олакшава поверење корисника у добијени редослед приоритета.
- **Отпорност кроз асинхрону архитектуру.** Подела на микросервise повезане преко RabbitMQ-а, уз ограничавање конкурентности семафором у сваком сервису (поглавље 4.4), омогућава да привремен пад или успорење једног сервиса не обори цео систем, и да се обрада репозиторијума са великим бројем фајлова паралелизује без неограниченог раста потрошње ресурса.
- **Модел свесан власништва над подацима.** Одвајање статуса/власништва посла (PostgreSQL, релационо, са страним везама ка кориснику) од неструктурираних резултата (MongoDB) омогућава како историју анализа по кориснику, тако и доследно, потпуно брисање података на захтев (поглавље 5.6.8), без дуплирања истог податка у две базе.

8.3 Ограничења решења

Ради целовитости и у складу са транспарентним приступом који је доследно примењен кроз цео рад, овде су сумирана најважнија ограничења тренутне имплементације, свако већ детаљније образложено у одговарајућем поглављу:

- ML Ranker Service је детерминистички, ручно калибрисан модел, не тренирани машински модел (поглавље 5.4.1); Suggestion Generator користи фиксне шаблоне, не LLM (поглавље 5.5.3).
- Analysis Service нема сопствени детектор SQL инјекције (ослања се искључиво на Semgrep, поглавље 5.3.6);
- Worker сервиси (Parser, Analysis, ML Ranker, Suggestion Generator) немају механизам поновног покушаја ни ред за неиспоручиве поруке — порука која не успе да се обради се тренутно губи (поглавље 5.2.6).
- Auth Service нема механизам за обнављање токена (поглавље 5.1.8); JWT се на клијентској страни чува у localStorage, а не у httpOnly колачићу (поглавље 5.7.6); CORS политика је тренутно пермисивна.
- Обрада кориснички достављених ZIP архива на API Gateway-у садржи позиве који на неисправном улазу доводе до панике уместо структурисаног одговора (поглавље 5.6.10).
- Auth, Suggestion Generator и API Gateway сервиси немају сопствене јединичне тестове; не постоје интеграциони тестови нити аутоматизован CI pipeline (поглавље 6.6).

- Систем не укључује Kubernetes оркестрацију, rate limiting на API Gateway-у, HTTPS терминацију, енкрипцију података у мировању, нити формалну усклађеност са регулативама попут GDPR-а или SOC 2 (поглавље 4.7).

Ниједно од наведених ограничења није последица превида — свако представља свесно донету одлуку о обиму, у корист комплетности архитектуре и тока података кроз систем у оквиру расположивог времена за израду дипломског рада.

8.4 Правци даљег развоја

На основу ограничења из претходног поглавља, даљи развој платформе могуће је усмерити у неколико праваца:

Квалитет рангирања и предлога

- Замена rule-based ML Ranker модела тренираним моделом (нпр. XGBoost или ONNX), при чему би обележја дефинисана у поглављу 5.4.3 (укључујући тренутно неискоришћена `params_count` и `file_size`) могла послужити као полазни скуп обележја, уз прикупљање скупа означених (labeled) примера проблема као основе за тренирање.
- Интеграција LLM модела у Suggestion Generator Service за генерисање контекстуалнијих предлога исправки, као алтернатива или допуна фиксним шаблонима.

Покривеност анализе

- Развој интерног детектора за SQL инјекцију, који би анализом тока података разликовао параметризоване упите од небезбедне конкатенације стрингова, уместо ослањања искључиво на `Semgrep`.
- Проширење и уједначавање скупа правила по језику.

Поузданост и отпорност

- Увођење реда за неиспоручиве поруке (dead-letter queue) и механизма поновног покушаја за поруке које `worker` сервиси не успеју да обраде.
- Замена `.unwrap()` позива у обради ZIP архива на API Gateway-у одговарајућом обрадом грешака.
- Увођење логике поновног повезивања и за API Gateway, по узору на `worker` сервисе (поглавље 5.6.9), ради отпорности на губитак везе са RabbitMQ-ом и након успешног старта сервиса.

Безбедност

- Механизам за обнављање JWT токена (refresh token) и одговарајуће `/api/auth/refresh` и `/api/auth/logout` руте.
- Замена чувања JWT токена у `localStorage` приступом заснованим на `httpOnly` колачићима.
- HTTPS терминација, енкрипција података у мировању, рестриктивнија CORS политика и формализација усклађености са регулативама попут GDPR-а.

Скалабилност и оперативна зрелост

- Kubernetes оркестрација као замена за Docker Compose у продукционом сценарију скалирања.
- Rate limiting на нивоу API Gateway-а.
- Аутоматизован CI pipeline који извршава постојећи скуп тестова при свакој измени кода.

Тестирање

- Јединични тестови за Auth, Suggestion Generator и API Gateway сервисе.
- Интеграциони тестови који покривају стварну размену порука преко RabbitMQ-а и ток података кроз више сервиса од краја до краја.

Корисничко искуство

- Замена периодичног поновног упита (polling) на страни Веб интерфејса и CLI алата механизмом заснованим на трајној вези (WebSocket или Server-Sent Events), ради бржег и ефикаснијег приказа напретка анализе.

Иако ниједан од наведених праваца није неопходан за функционисање система у обиму демонстрираном у овом раду, сваки представља природан следећи корак ка платформи спремној за продукционо окружење са више корисника и већим обимом кода.

Списак слика

Слика 1	Преглед архитектуре: клијенти, API Gateway, RabbitMQ pipeline и базе података	14
Слика 2	Пријавна страница Веб интерфејса	57
Слика 3	Dashboard са унетим примером кода пре покретања анализе	59
Слика 4	Праћење напретка анализе	59
Слика 5	Преглед резултата	60
Слика 6	Бочна трака са историјом анализа корисника	61

Списак листинга

Листинг 1	Хеш лозинке и упис корисника (auth_service/src/main.rs)	21
Листинг 2	Креирање JWT токена (auth_service/src/jwt.rs)	22
Листинг 3	Трејт LanguageParser и фабричка функција (parser_service/src/parsers/mod.rs)	24
Листинг 4	Иницијализација Python парсера (parser_service/src/parsers/python.rs) .	25
Листинг 5	Трејт Detector (analysis_service/src/detectors/mod.rs)	27
Листинг 6	Детекција Н+1 упита по увучености тела петље, скраћено (analysis_service/src/detectors/performance.rs)	29
Листинг 7	Позив Semgrep-а као екстерног процеса (analysis_service/src/semgrep.rs)	31
Листинг 8	Правило приоритета при дедупликацији, скраћено (analysis_service/src/dedup.rs)	32
Листинг 9	Формула рангирања проблема, скраћено (ml_ranker_service/src/ml_engine.rs)	36
Листинг 10	Препознавање типа проблема и генерисање предлога по језику, скраћено (suggestion_generator_service/src/suggestions.rs)	38
Листинг 11	Ажурирање напретка и статуса посла, извршава се по сваком обрађеном фајлу (suggestion_generator_service/src/main.rs)	40
Листинг 12	Redis кеширање комплетних резултата посла, скраћено (api_gateway/src/routes.rs)	44
Листинг 13	Образац тестних података “подразумевано па изузетак” (analysis_service/src/detectors/smells.rs)	52
Листинг 14	Гранични тест услова “> 50 линија” (analysis_service/src/detectors/smells.rs)	53
Листинг 15	Регресиони тест за исправљен образац препознавања AWS креденцијала (analysis_service/src/detectors/security.rs)	54
Листинг 16	Тест монотоности хијерархије категорија (ml_ranker_service/src/ml_engine.rs)	55
Листинг 17	Илустративан ток извршавања CLI алата над примером из поглавља 7.3	62

Списак табела

Табела 1	Упоредни преглед алата за статичку анализу	5
Табела 2	Портови сервиса (docker-compose)	11
Табела 3	Преглед микросервиса платформе RepoOptimizer	15
Табела 4	Структуре података Auth Service-а	20
Табела 5	Руте Auth Service-а	20
Табела 6	Кључне структуре података Parser Service-а	26
Табела 7	Прагови за code smell проблеме	28
Табела 8	Обележја коришћена при рангирању проблема	34
Табела 9	Тежине категорија проблема (извод)	35
Табела 10	Покривене категорије проблема и оцена утицаја предлога	39
Табела 11	Структура Suggestion (уписује се у MongoDB колекцију suggestions) .	40
Табела 12	Руте API Gateway-а	42
Табела 13	Команде CLI алата	49
Табела 14	Преглед јединичних тестова по сервису	51

Списак коришћених скраћеница

Скраћеница	Опис
AMQP	Advanced Message Queuing Protocol (протокол за размену порука)
API	Application Programming Interface (апликациони програмски интерфејс)
AST	Abstract Syntax Tree (апстрактно синтаксно стабло)
AWS	Amazon Web Services (Амазон веб сервиси)
CI/CD	Continuous Integration / Continuous Delivery (континуирана интеграција / континуирана испорука)
CLI	Command Line Interface (команднолинијски интерфејс)
CORS	Cross-Origin Resource Sharing (размена ресурса између извора различитог порекла)
DTO	Data Transfer Object (објекат за пренос података)
GDPR	General Data Protection Regulation (Општа уредба о заштити података)
HTTP/HTTPS	HyperText Transfer Protocol (Secure) (протокол за пренос хипертекста)
I/O	Input/Output (улаз/излаз)
JSON	JavaScript Object Notation (формат за размену података)
JWT	JSON Web Token (безбедносни токен заснован на JSON формату)
LLM	Large Language Model (велики језички модел)
ML	Machine Learning (машинско учење)
ONNX	Open Neural Network Exchange (отворен формат за размену модела машинског учења)
QL	Query Language (упитни језик, у контексту CodeQL)
RAM	Random Access Memory (меморија са случајним приступом)
REST	Representational State Transfer (скуп правила за комуникацију клијент-сервер)
SOC 2	Service Organization Control 2 (стандард усклађености за безбедност података)
SPA	Single Page Application (једнострана веб апликација)

SQL	Structured Query Language (структурирани упитни језик)
TLS	Transport Layer Security (безбедност транспортног слоја)
URL	Uniform Resource Locator (јединствени локатор ресурса)
UI	User Interface (кориснички интерфејс)
UUID	Universally Unique Identifier (универзално јединствени идентификатор)
UTF-8	Unicode Transformation Format – 8-bit (формат кодирања текста)
XSS	Cross-Site Scripting (напад убацивањем скрипти кроз веб страницу)
YAML	YAML Ain't Markup Language (формат за серијализацију података)
ZIP	назив формата за компресију и архивирање фајлова

Списак коришћених појмова

Појам	Објашњење
Асинхрони рад	Рад који се изводи независно од главног тока извршавања, омогућавајући наставак других операција без чекања на његов завршетак.
Микросервис	Независно развијана и покретана компонента система која имплементира уско дефинисан део пословне логике и комуницира са осталим компонентама преко јасно дефинисаног интерфејса.
Message broker (посредник за размену порука)	Инфраструктурна компонента која прихвата поруке од произвођача и прослеђује их потрошачима, омогућавајући лабаво повезивање између компоненти система.
Connection pool (скуп конекција)	Унапред припремљен скуп отворених конекција ка бази података које се поново користе за више узастопних упита, ради избегавања трошка отварања нове конекције за сваки упит.
Семафор (concurrency semaphore)	Синхронизациони механизам који ограничава број задатака који се истовремено извршавају, додељивањем и ослобађањем ограниченог броја “дозвола”.
Кеш (cache)	Привремено складиште већ израчунатих или добављених података, коришћено ради бржег поновног приступа истим подацима.
Dead-letter queue (ред за неиспоручиве поруке)	Посебан ред у који се преусмеравају поруке које потрошач није успео да обради, ради касније анализе или поновног покушаја обраде.

Polling (периодично поновно упитивање)	Начин праћења стања у коме клијент у правилним временским размацама поново шаље захтев серверу да провери да ли је дошло до промене, уместо да сервер сам обавести клијента.
Дедупликација	Поступак препознавања и уклањања дуплираних записа који представљају исти стварни ентитет, добијених из различитих извора.

Биографија

Милан Лазаревић основне академске студије на смеру Софтверско инжењерство и информационе технологије уписао је 2022. године на Факултету техничких наука у Новом Саду, које завршава 2026. године изградом овог рада. Током студија се посебно интересовао за дистрибуиране системе, што је утицало и на избор теме завршног рада — пројектовање и имплементацију микросервисне платформе за статичку анализу изворног кода.

Литература

- [1] SonarSource, “Code Quality, Security & Static Analysis Tool with SonarQube.” Accessed: July 27, 2026. [Online]. Доступно на <https://www.sonarsource.com/products/sonarqube/>
- [2] Semgrep, Inc., “Semgrep Code overview.” Accessed: July 27, 2026. [Online]. Доступно на <https://semgrep.dev/docs/semgrep-code/overview>
- [3] GitHub, “About CodeQL.” Accessed: July 27, 2026. [Online]. Доступно на <https://codeql.github.com/docs/codeql-overview/about-codeql/>
- [4] Rust Foundation, “Rust Programming Language.” Accessed: July 27, 2026. [Online]. Доступно на <https://www.rust-lang.org/>
- [5] Actix, “actix_web – Rust.” Accessed: July 27, 2026. [Online]. Доступно на <https://docs.rs/actix-web>
- [6] Tree-sitter, “tree-sitter: An incremental parsing system for programming tools.” Accessed: July 27, 2026. [Online]. Доступно на <https://github.com/tree-sitter/tree-sitter>
- [7] RabbitMQ, “AMQP 0-9-1 Model Explained.” Accessed: July 27, 2026. [Online]. Доступно на <https://www.rabbitmq.com/tutorials/amqp-concepts>
- [8] Meta, “React – The library for web and native user interfaces.” Accessed: July 27, 2026. [Online]. Доступно на <https://react.dev/>
- [9] Vite, “Vite.” Accessed: July 27, 2026. [Online]. Доступно на <https://vitejs.dev/>
- [10] Microsoft, “Monaco Editor.” Accessed: July 27, 2026. [Online]. Доступно на <https://microsoft.github.io/monaco-editor/>